SIMATS ENGINEERING

Department of Information Technology

DSA03 – Natural Language Processing

ASSIGNMENT 1 – CO1

Model Answer with Experimental Evidence

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Student Name	Murthi Sreevardhan	**Register Number**	192472405
Department / Section	CSE-AI	**Academic Year / Semester**	3rd year
Faculty Name	Dr.M.Ramkumar	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	2-11-26	**Date of Evaluation**	

MARK SHEET (For Evaluator Use)

Task	Task Title	Max	Marks Awarded	Remarks
1	Frame the Problem: Models, Algorithms and Levels of Analysis	3		
2	Basic Regular Expression Patterns	4		
3	Regular Expressions for Tokenisation and Normalisation	4		
4	Finite State Automata: Design, Non-determinism and Equivalence	5		
5	Inflectional Morphology	3		
6	Derivational Morphology	4		
7	Finite-State Morphological Parsing	5		
8	Orthographic Rules and Two-Level Morphology	4		
9	The Porter Stemmer	5		
10	Implement and Test the Pipeline	3		
11	Compare, Recommend and Defend the Normalisation Design	2		
12A	Impact Analysis	2		
12B	Evaluation of the Original Design	2		
12C	Adaptation or Redesign	2		
12D	Evidence-Based Defence	2		
	TOTAL	50		

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence Expected	Status (Yes / Partly / No)	Evaluator Note
1	Levels of analysis, three model families, four ambiguities, search resources, labelled pipeline diagram		

Task	Evidence Expected	Status (Yes / Partly / No)	Evaluator Note
2	Pattern inventory (13 types) with match specification and per-pattern TP/FP/FN error analysis		
3	Working regex tokenizer and normaliser, 3 maximal-munch examples, full before/after record, limits of regularity		
4	Four DFAs (diagram + table), NFA with ε, subset construction, minimisation, Thompson and state-elimination, pumping lemma		
5	Inflectional rules and paradigms, irregular list, type-to-lemma ratio and retrieval failures explained by inflection		
6	Affix inventory (41), derivational families, non-compositionality, productivity counts, conflation position		
7	FST specification with continuation classes, 5 traces, generation, ambiguity		
8	Rewrite rules, cascade and order dependencies, two-level contrast, 22-form test with error accounting		
9	Porter measure and steps traced, stem vs lemma, over/under-stemming rates, three-condition retrieval experiment		
10	Python implementation, stage table, tests and plots		
11	Comparison of three strategies, recommendation, effort and break-even discussion		
12	Impact analysis, original-design evaluation (3 traces + degradation), adaptation and evidence-based defence		

IMPLEMENTATION EVIDENCE CHAIN (Task 10)

Evidence Item	Present (Yes / No)	Evaluator Observation
Corpus documentation (generator, seed, record types, token counts)		
Pattern inventory and per-pattern results		
Automata: DFA tables, NFA trace, subset construction, minimisation outputs		
Lexicon, affix tables, irregular lists		
Transducer traces matching code output		
Porter hand-traces matching code output		
Evaluation data: 40 queries with judgements, 200 annotated forms		
Stage table (items, matched, rejected, run-time) and plots		
Day-4 noisy and code-mixed samples with before/after measurements		

SUBMISSION CHECKLIST

Requirement	Yes	Partly	No
Corpus ≥ 500 records, ≥ 25,000 word tokens, ≥ 4 record types	☐	☐	☐
≥ 12 entity patterns each with positive and negative specification	☐	☐	☐
≥ 4 automata with ≥ 6 states, one NFA with ε-transitions	☐	☐	☐
Lexicon ≥ 300 stems, ≥ 20 inflectional and ≥ 15 derivational affixes	☐	☐	☐
≥ 40 queries with judgements and ≥ 200 annotated word forms	☐	☐	☐
Manual effort and cost accounted for	☐	☐	☐

Requirement	Yes	Partly	No
Python code uses standard library and NLTK only	☐	☐	☐
Day-4 challenge answered under Impact, Evaluation, Adaptation and Defence	☐	☐	☐

DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 2-11-2026

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.
- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4): The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language.

EXPERIMENTAL SET-UP USED FOR ALL ANSWERS

Every number in this document was produced by the Python programs submitted with it (standard library and NLTK only; NLTK is used solely to cross-check my own Porter implementation). The hospital data are not available, so the corpus is **synthetic**: it is generated from clinical templates with a fixed random seed (2026), so it is fully documented and reproducible. The text is grammatical but not clinically meaningful, and the entities, inflected forms and out-of-lexicon words were deliberately seeded with the phenomena the assignment asks about. The figures therefore measure how each formalism copes with those phenomena, not real hospital accuracy; Section 11 states what this does and does not prove.

Table 1: Resources built for this assignment

Resource	Requirement	What I built (measured)
Corpus	≥ 500 records, ≥ 25,000 tokens, ≥ 4 types	520 records (130 each: discharge summary, prescription, operation note, nursing observation); 52,595 alphabetic word tokens, 72,110 tokenizer tokens, 692 word types; 7,268 labelled entity mentions

Resource	Requirement	What I built (measured)
Patterns	≥ 12 entity types	13 regular expressions (patient ID, admission date, discharge date, drug + strength, dose frequency, laboratory value, blood pressure, temperature, ICD code, ward/bed, contact number, age, symptom duration)
Automata	≥ 4 automata, ≥ 6 states, one NFA	4 DFAs (ICD code 10 states, blood pressure 8, ward/bed 8, dose frequency 6); temperature NFA with ε-moves (11 states); Thompson NFA (28 states)
Lexicon	≥ 300 stems	507 stems (253 nouns, 167 verbs, 104 adjectives) plus 33 bound roots; compiled lexicon FST has 14,034 states and 2,005 lexical entries
Affixes	≥ 20 inflectional, ≥ 15 derivational	24 inflectional realisations (Table in Task 5); 25 derivational suffixes + 16 prefixes = 41; 168 lexicon-licensed derivations
Irregular forms (listed separately)	—	34 nouns, 34 verbs, 4 adjectives, 11 derived words
Evaluation data	≥ 40 queries, ≥ 200 annotated forms	40 queries with record-level relevance judgements (6 categories); 200 word forms with gold lemma and features (taken from the generator, which knows the lexeme behind every inflected form)

Relevance judgements. Each record carries the set of concepts (lemmas) the generator put into it, including words in fixed template phrases such as "On examination". A record is relevant to a query when it carries the query concept; for the seven "derivational relative" queries the judgement also accepts the clinically equivalent derived concept (operate → operation, treat → treatment). This is a documented simplification of real clinician judgement.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3 Marks

Guiding Enquiry — Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis and state, with an example from your corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish state machines, formal rule systems and probabilistic models; name one task per family and say why the other two would be wrong.
- Explain why ambiguity is the central problem and identify at least four distinct ambiguities (lexical, morphological, segmentation, abbreviation).
- Identify where search appears (state-space search in automata, backtracking in matching) and the resources each consumes.
- Set out the pipeline as a labelled diagram naming input, output and formalism of every stage.

Evidence Expected — A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

Ans:

1. Levels of linguistic analysis

Table 2: Levels of analysis and what the record-search problem needs

Level	Needed?	Example from my corpus	Decision
Phonology	No (typed text)	Not applicable to written records. It re-appears only in Task 12: Romanised Tamil "irundhuchu / irunthuchu / irundhuchi" are spelling variants of one pronunciation.	Ignore in Tasks 1–11; handle by edit distance in Task 12.
Morphology	Yes – central	"fractures, fractured, fracturing" must reach the same query "fracture"; without conflation only 56.5% of relevant records are found (Task 9).	Core of the pipeline.

Level	Needed?	Example from my corpus	Decision
Syntax	Mostly no	"No pruritus or oedema was observed" – negation scope matters clinically but a bag-of-lemmas search does not use it.	Ignore; list as limitation.
Semantics	Partly	"drain" is a device in "the drain was sutured" and a procedure in "drain the abscess"; "discharge" is an event or a wound symptom.	Keep all senses; do not resolve.
Pragmatics	No	Imperative "Avoid pain; treat the fracture" in prescriptions is an instruction, not an assertion about the patient.	Safely ignored for retrieval.
Discourse	No	"She gave the tablet" refers to a patient named earlier in the record.	Safely ignored; each record is one search unit.

2. Three model families

Table 3: One hospital task per model family

Family	Task it suits best	Why the other two are wrong for it
State machines (regular expressions, automata, transducers)	Recognising and validating structured entities such as blood pressure "154/98 mmHg" and, in transducer form, relating "fractures" to "fracture +N +PL".	The format is a fixed finite pattern, so a rule system (context-free grammar) is more power than needed and slower to write; a probabilistic model would need labelled examples and could still accept an impossible "1200/80".
Formal rule systems (rewrite rules, grammars)	Spelling changes at morpheme boundaries: y → ie before -s ("carry^s" → "carries"), e-deletion ("fracture^ing" → "fracturing").	The rules are exhaustive and deterministic, so a probabilistic model would only blur them and needs annotated data I do not have; a bare state machine can run the rules but cannot state them readably or order them.
Probabilistic models	Choosing between analyses and languages: "fractured" as past vs past participle, "OD" as once daily vs right eye, or an English vs Romanised-Tamil token.	No automaton or rule can say which analysis is more likely in context; they can only list the options, and a rule-based choice would be brittle.

3. Ambiguity – the central problem

Every stage of the pipeline has to choose between readings of the same string. In my corpus 8,222 of 52,595 word tokens (15.6%) receive more than one analysis from the transducer, and the pattern layer makes 701 false-positive entity matches that are all ambiguity errors.

Table 4: Four distinct kinds of ambiguity found in the corpus

Kind	Corpus evidence	Why it is ambiguous
Lexical	"drain", "burn", "cut", "saw": burns → burn +N +PL / burn +V +3SG; "saw" → saw +N +SG / see +V +PAST	One spelling, two lexemes with different parts of speech.
Morphological	leaves → leaf +N +PL / leave +V +3SG; left → leave +V +PAST / +PSP / left +A; fractured → +PAST / +PSP	One suffix realises several features, or one surface form fits several stems.
Segmentation	"12/08" matched as blood pressure 49 times where it was a score or short date ("10/10", "12/08"); "c/o", "y/o" and "mg/dL" are one token each, but "B12" in "Vitamin B12" looks like an ICD code	The boundary and the type of a token cannot be decided from its characters alone.
Abbreviation	"OD" (once daily) was matched 84 times where it meant right eye ("Eye exam: OD 6/6"); "HS", "Ca", "MS" are similar	The same letters expand to different clinical concepts depending on context.

4. Where search appears and what it costs

Table 5: Search in the unit and the resources it consumes (measured on 520 records)

Search	Where	Time	Memory	Annotated data	Expert effort
Backtracking	Regex matching of the 13 patterns (Python re)	0.23 s for 520 records; worst case exponential for nested quantifiers, linear for mine	Pattern text and a backtrack stack	None for matching; truth labels for evaluation	High: writing and testing each pattern
State-space search	NFA simulation (set of live states) and subset construction	O(|w|·|Q|) per string; subset construction reached 10 of 2^11 = 2,048 possible subsets	One set of states	None	Medium: designing the automaton
Depth-first search over arcs	Running the lexicon FST on a form	0.13 s for 50,013 tokens; path length = word length (acyclic)	14,034 states and 14,033 arcs	200 forms to test	Very high: 507 stems, affixes and irregulars written by hand
Rule-ordered rewriting	Spelling-rule cascade (7 rules)	Linear in word length per rule	Negligible	22 test forms	Medium: rules and exceptions
Rule application	Porter stemmer (5 steps)	0.42 s for 50,013 tokens	Suffix tables only	None	Low: algorithm exists

5. The pipeline



Figure 1: Pipeline built in Tasks 2–10 (formalism of every stage in brackets)

Table 6: Input, output and formalism of every stage

Stage	Input	Output	Formalism
1 Tokenizer	Raw record string	Typed tokens (PID, DATE, BP, NUM, UNIT, ABBR, HYPH, WORD, PUNCT)	Regular expression with maximal munch
2 Entity recognisers	Record string	Entity spans of 13 types; strict validation for ICD, BP, ward, frequency	Regular expressions, DFAs
3 Normaliser	Typed tokens	Case-folded tokens, expanded abbreviations, ISO dates, canonical units	Substitution rules (regular relations)
4 Morphological analyser	Word token	All analyses: lemma + part of speech + features (+ affixes)	Lexicon FST composed with spelling rules
5 Stemmer	Word token	Stem (no lexicon)	Porter rule cascade

TASK 2 – Basic Regular Expression Patterns	4 Marks

Guiding Enquiry — What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators and give one minimal clinical example of each.
- Write and document a regular expression for each entity type, stating what it must match and must not match.
- Demonstrate greedy and non-greedy matching on a sentence with two dosage expressions and explain why the spans differ.
- Explain precedence and grouping with a worked example of a misplaced disjunction.
- Evaluate every pattern over the corpus (TP, FP, FN) and name the linguistic cause of each error class.

Evidence Expected — A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

Ans:

1. Basic operators

Table 7: Basic regular-expression operators with a minimal clinical example

Operator	Meaning	Clinical example	Matches
Concatenation	x then y	HX-	“HX-” in HX-2025-001234
Disjunction |	x or y	OD|BD|TDS	any of the three frequency codes
Kleene star *	zero or more	\d*	“”, “5”, “500”
Kleene plus +	one or more	\d+mg	“5mg”, “500mg” (not “mg”)
Optionality ?	zero or one	\d+\s?mg	“500mg” and “500 mg”
Character class [ ]	one of a set	[CF]	“C” or “F” in a temperature
Range	consecutive set	[A-D]	ward letter A, B, C or D
Negated class [^]	anything except	[^0-9]	any non-digit, e.g. the “/” in 120/80
Anchors ^ $ \b	position, not character	\bOD\b	“OD” as a whole word, not inside “ODOR”
Counters {n,m}	repeat n to m times	\d{6}	the six-digit serial of a patient ID
Grouping ( )	treat as unit	(mg|g)	unit alternation
Wildcard .	any character	Hb.*dL	text from Hb to dL

2. The 13 patterns with match specification

Table 8: Pattern inventory: regex, must match, must not match

Entity	Regular expression (Python re)	Must match	Must not match
Patient ID	\bHX(?:-\d{4}-|/\d{2}/)\d{6}\b	HX-2024-001234, HX/24/001234	HX 2024 001234 (spaces); 2024-001234
Admission date	(?:(?<=[Aa]dmitted on)|(?<=DOA:)|(?<=Date of admission))(?:\d{2}/\d{2}/\d{4}|\d{2}-(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)-\d{4}|\d{4}-\d{2}-\d{2}|\d{1,2} (?:January|February|March|April|May|June|July|August|September|October|November|December) \d{4})	date after “Admitted on”: 12/03/2025, 12-Mar-2025, 2025-03-12, 12 March 2025	date after “Discharged on”; a bare date with no cue
Discharge date	(?:(?<=[Dd]ischarged on)|(?<=discharge on)|(?<=DOD:)|(?<=Date of discharge))(?:\d{2}/\d{2}/\d{4}|\d{2}-(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)-\d{4}|\d{4}-\d{2}-\d{2}|\d{1,2} (?:January|February|March|April|May|June|July|August|September|October|November|December) \d{4})	same date formats after “Discharged on” / “discharge on”	date after “Admitted on”
Drug + strength	\b(?:(?:Tab|Cap|Inj|Syp)\.\s)?[A-Za-z]{4,}\s\d+(?:\.\d+)?\s?(?:mg|mcg|g|ml|IU|units)(?![/\w])	Tab. Paracetamol 500 mg, losartan 5mg, Inj. Insulin 40 IU	“Hb 11 g/dL” (lab unit), word shorter than 4 letters

Entity	Regular expression (Python re)	Must match	Must not match
Dose frequency	\b(?:OD|BD|TDS|QID|HS|SOS|\d-\d-\d|(?:once|twice) daily|three times a day|every \d{1,2} hours)\b	OD, BD, TDS, QID, HS, SOS, 1-0-1, twice daily, every 8 hours	“ODOR”, “1-0” (incomplete)
Lab value + unit	\b[A-Z][A-Za-z0-9]{1,10}:?\s\d[\d,]*(?:\.\d+)?\s?(?:g/dL|/cumm|mg/dL|mmol/L|mg/L|lakh/cumm)(?!\w)|\bSpO2:?\s\d{2,3}%	Hb 11.2 g/dL, WBC 12,500 /cumm, Creatinine 1.4 mg/dL, SpO2 97%	a drug strength “Dose 5 mg” (unit mg alone)
Blood pressure	(?<![/\d.-])\d{2,3}\s?/\s?\d{2,3}(?:\s?mmHg)?(?![/\d])	120/80, 154 / 98 mmHg	part of a date (15/01/2026), 1200/80, 12/8
Temperature	\b\d{2,3}(?:\.\d)?\s?(?:°\s?[CF]|degrees\s[CF]|[CF])\b	37.2°C, 98.6 F, 101.4 degrees F	“Bed 12C” (letter-suffixed bed), “vitamin C”
ICD code	\b[A-TV-Z]\d{2}(?:\.\d{1,4}[A-Z]?)?\b	S72.001A, I10, E11.9, K35.80	lower-case codes, “HX-2025-…”, one-digit codes
Ward and bed	\b(?:(?:Ward|W)\s?[1-9][A-D](?:,\s|\s|/)(?:Bed|B)\s?\d{1,2}[A-D]?|(?:ICU|HDU)\sBed\s\d{1,2})\b	Ward 4B, Bed 12; W4B/B12; ICU Bed 3	Ward-4B-Bed-12 (hyphenated), “Bed 12” alone

Entity	Regular expression (Python re)	Must match	Must not match
Contact number	(?<![\d-])(?:\+91\s\d{5}\s\d{5}|\d{10}|0\d{2,3}-\d{4}\s\d{4})(?![\d-])	+91 98765 43210, 9876543210, 044-2345 6789	11-digit strings, digits inside longer numbers
Age	\b(?:\d{1,2}(?:\s?y/o|-year-old|\s?yrs?|\s?years|Y/[MF])|aged\s\d{1,2})\b	45 y/o, 45-year-old, 45 yrs, 45Y/M, aged 45	durations such as “for 5 yrs”
Symptom duration	\b(?:for(?:\sthe\spast)?\s\d{1,2}\s(?:days?|weeks?|months?|yrs?)|since\s\d{1,2}\s(?:days?|weeks?|months?)|x\s\d{1,2}\s(?:days?|weeks?))\b	for 3 days, since 2 weeks, x 5 days, for the past 2 months	ages (“45 yrs”), a bare “3 days ago”

3. Greedy versus non-greedy matching

Sentence: “Give Paracetamol 500 mg in the morning and Ibuprofen 400 mg at night.”. Pattern: greedy [A-Z][a-z]+\s\d+.*mg against lazy [A-Z][a-z]+\s\d+.*?mg.

Quantifier	Span returned	Characters
Greedy .*	“Paracetamol 500 mg in the morning and Ibuprofen 400 mg”	5–59
Non-greedy .*?	“Paracetamol 500 mg”	5–23

Both patterns first match at “Paracetamol 500”. The greedy .* consumes the whole rest of the sentence and then **backtracks one character at a time** until the literal “mg” can match, so it stops at the last “mg” and swallows both dosages. The lazy .*? consumes as little as possible and extends only when “mg” fails, so it stops at the first “mg”. For extracting one dosage per drug, the lazy form (or better a character class such as [^.;]*?) is correct.

4. Precedence and grouping

Precedence from highest to lowest is: grouping, counters/star/plus/optional, concatenation, then disjunction. Disjunction binds loosest, so it splits the whole expression unless it is grouped.

Expression	Intended language	Actual behaviour on “ODOR noted; XBD entered.”
\bOD|BD\b	“OD” or “BD” as whole words	Parsed as (\bOD)|(BD\b): matches “OD” and “BD” – the OD inside “ODOR” and the BD at the end of “XBD”
\b(?:OD|BD)\b	same	No match, which is correct

The misplaced disjunction silently changes the denoted language: the first expression accepts every word beginning with OD and every word ending with BD, so it would label “ODOR” as a dose frequency without any error message.

5. Evaluation over the corpus

I compared every match with the generator’s entity spans (exact span and type). Over 7,268 labelled mentions the 13 patterns obtain **6,746 TP, 701 FP, 522 FN** (micro precision 90.6%, recall 92.8%).

Table 9: Per-pattern error analysis on the full corpus

Entity	TP	FP	FN	P	R	Cause of false positives	Cause of false negatives
Patient ID	498	0	22	1.000	0.958	—	Space-separated variant “HX 2023 …” (formatting variation not in the pattern).

Entity	TP	FP	FN	P	R	Cause of false positives	Cause of false negatives
Admission date	248	0	12	1.000	0.954	—	Two-digit-year dotted dates (12.03.25) are not in the date alternation.
Discharge date	189	0	17	1.000	0.917	—	Same dotted two-digit-year dates.
Drug + strength	928	73	0	0.927	1.000	A common noun before a number is read as a drug ("Dose 5 mg", "dose 500 mg") – lexical ambiguity.	—
Dose frequency	790	84	138	0.904	0.851	"OD" meaning right eye – abbreviation ambiguity.	Dotted or q-notation (b.d., t.i.d., q8h) – variant notation.
Lab value + unit	402	0	73	1.000	0.846	—	Single-letter analyte "K" (pattern wants ≥ 2 characters) and garbled units (gm%, mg/dlx) – orthographic variation.
Blood pressure	390	127	0	0.754	1.000	Two-digit ratios that are not pressures: score 10/10, short date 12/08 – segmentation/type ambiguity.	—
Temperature	363	45	27	0.890	0.931	Bed labels such as "12C" and the stray "38 C" – unit letter shared with another code.	Abbreviated unit "deg F".
ICD code	238	151	4	0.612	0.983	Letter + two digits also names vitamin B12, grade A12, lot T20 and bed B12 – shape ambiguity.	Lower-case codes ("i10").
Ward and bed	369	0	21	1.000	0.946	—	Hyphenated "Ward-4B-Bed-12".
Contact number	540	41	78	0.929	0.874	A 10-digit accession number is a valid-looking phone – shape ambiguity.	Hyphenated mobile (98765-43210) and bracketed landline.
Age	372	180	18	0.674	0.954	"for 5 yrs" is a duration, not an age – same unit word.	"yo" abbreviation.
Symptom duration	1419	0	112	1.000	0.927	—	Shorthand "n/7" (days).

Two error families dominate. **Shape ambiguity** (BP, ICD, AGE, PHONE, DRUG) lowers precision because several clinical concepts share one character shape, which no regular expression can separate without context. **Variant notation** (FREQ, LAB, DATE, PID, WARD) lowers recall because every unforeseen way of writing the same thing needs a new alternative. The first needs context or a probabilistic decision; the second needs more patterns and is where the automata and normaliser of Tasks 3–4 help.

TASK 3 – Regular Expressions for Tokenisation and Normalisation	4 Marks

Guiding Enquiry — How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify the tokenizer as a single regular expression and justify each decision.
- Apply maximal-munch tokenisation and show on at least three examples how it differs from a whitespace split.
- Build substitution rules that fold case, expand at least thirty abbreviations and canonicalise dates and units; show a full record before and after.
- Identify at least three irreversible normalisation decisions.
- Give one requirement that no regular expression can express.

Evidence Expected — A working regular-expression tokenizer and normaliser with worked examples and a clear

statement of the limits of regularity.

Ans:

1. The tokenizer as one regular expression

The tokenizer is the union of twelve named alternatives, (?P<PID>…)|(?P<DATE>…)|…|(?P<SYM>\S). Python's re returns the first alternative that matches, not the longest, so I apply **maximal munch** explicitly: at each position every alternative is tried and the longest match wins; equal lengths are resolved by the order in the table. That makes the tokenizer a regular language recogniser with deterministic longest-match semantics.

Table 10: Token classes, expressions and justification

Class	Expression	Why this decision
PID	HX(?:-\d{4}-|/\d{2}/|\s\d{4}\s)\d{6}	Identifier must stay whole; three separator variants.
DATE	(?:\d{2}/\d{2}/\d{4}|\d{2}-(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)-\d{4}|\d{4}-\d{2}-\d{2}|\d{1,2} (?:January|February|March|April|May|June|July|August|September|October|November| December) \d{4})|\d{2}\.\d{2}\.\d{2}	Four formats plus dotted two-digit year, so they can be rewritten to ISO later.
PHONE	(?<![\d-])(?:\+91\s\d{5}\s\d{5}|\d{5}-\d{5}|\d{10}|0\d{2,3}-\d{4}\s\d{4}|\(0\d{2,3}\)\s\d{8})(?![\d-])	Ten digits with optional +91 or STD code; the look-around stops it eating digits of longer numbers.
BP	\d{2,3}\s?/\s?\d{2,3}(?![\d/])	Systolic/diastolic is one measurement; spaces around "/" allowed.
ICD	[A-TV-Z]\d{2}(?:\.\d{1,4}[A-Z]?)?(?![\w])	Letter + 2 digits + optional decimal part; the look-ahead stops "B12x" being a code.
NUM	\d+(?:,\d{3})*(?:\.\d+)?	Decimals and thousands separators (12,500) are single numbers.
UNIT	°\s?[CF]|%|/cumm|(?:mg|mcg|g|ml|IU|mmHg|mmol|dL|cumm)(?:/(?:dL|L|cumm))?	Units glued to numbers (500mg, 37.2°C) become NUM + UNIT, so "500mg" and "500 mg" give identical tokens.

Class	Expression	Why this decision
ABBR	(?:[A-Za-z]\.){2,}|(?:Tab|Cap|Inj|Syp|Dr|Mr|Mrs|No)\.|[A-Za-z]/[A-Za-z]{1,2}(?![\w/])	Abbreviations with full stops (b.d., Tab.) and slash abbreviations (c/o, y/o) must not be split at "." or "/".
HYPH	[A-Za-z]+(?:-[A-Za-z]+)+	Hyphenated clinical terms (post-operative, follow-up) are one word for later morphology.
WORD	[A-Za-z]+(?:'[a-z]+)?	Alphabetic words with optional clitic.
PUNCT	[.,;:()/]	Sentence punctuation as separate tokens.
SYM	\S	Fallback so no character is lost.

2. Maximal munch against whitespace splitting

Table 11: Three examples: naive split versus maximal munch

Input	Whitespace split	Maximal munch (token ␣ token)
BP 154 / 98 mmHg, Temp 39.4°C,	[BP] [154] [/] [98] [mmHg,] [Temp] [39.4°C,]	[BP]:WORD [154 / 98]:BP [mmHg]:UNIT [,]:PUNCT [Temp]:WORD [39.4]:NUM [°C]:UNIT [,]:PUNCT
Tab. Paracetamol 500mg b.d. since 12/03/2025.	[Tab.] [Paracetamol] [500mg] [b.d.] [since] [12/03/2025.]	[Tab.]:ABBR [Paracetamol]:WORD [500]:NUM [mg]:UNIT [b.d.]:ABBR [since]:WORD [12/03/2025]:DATE [.]:PUNCT
Hb 11.2 g/dL, WBC 12,500 /cumm, post-operative follow-up.	[Hb] [11.2] [g/dL,] [WBC] [12,500] [/cumm,] [post-operative] [follow-up.]	[Hb]:WORD [11.2]:NUM [g/dL]:UNIT [,]:PUNCT [WBC]:WORD [12,500]:NUM [/cumm]:UNIT [,]:PUNCT [post-operative]:HYPH [follow-up]:HYPH [.]:PUNCT

Whitespace splitting leaves punctuation glued to words ("mmHg,", "39.4°C,"), keeps "154 / 98" as three fragments and glues "500mg" to its unit. Over the corpus the whitespace split gives 57,585 tokens and maximal munch gives 72,110 (25.2% more), and every extra token is a unit, number or punctuation mark that the search index no longer confuses with a word.

Table 12: Token classes produced on the corpus

WORD	PUNCT	NUM	UNIT	ABBR	PHONE	PID	BP	SYM	DATE	ICD	HYPH
47,560	12,962	5,606	1,949	736	659	520	517	492	466	389	254

3. Normalisation by substitution rules

The normaliser applies four families of rules to the typed tokens: case folding, abbreviation expansion, date canonicalisation and unit canonicalisation. Identifiers and ICD codes are protected from case folding.

Table 13: Abbreviation expansion rules (54 rules, ≥ 30 required)

Abbr.	Expansion	Abbr.	Expansion	Abbr.	Expansion
pt	patient	im	intramuscular	hx	history
c/o	complaining of	sc	subcutaneous	htn	hypertension
h/o	history of	po	by mouth	dm	diabetes mellitus

Abbr.	Expansion	Abbr.	Expansion	Abbr.	Expansion
o/e	on examination	wbc	white blood cell count	copd	chronic obstructive pulmonary disease
y/o	year old	hb	haemoglobin	uti	urinary tract infection
bd	twice daily	bp	blood pressure	sob	shortness of breath
b.d.	twice daily	hr	heart rate	crp	c-reactive protein
tds	three times daily	rr	respiratory rate	mi	myocardial infarction
t.i.d.	three times daily	spo2	oxygen saturation	tab.	tablet
qid	four times daily	ecg	electrocardiogram	cap.	capsule
od	once daily	ct	computed tomography	inj.	injection
o.d.	once daily	mri	magnetic resonance imaging	syp.	syrup
hs	at bedtime	icu	intensive care unit	dr.	doctor
sos	if needed	ot	operation theatre	temp	temperature
prn	as needed	opd	outpatient department	yrs	years
stat	immediately	dx	diagnosis	yr	year
npo	nothing by mouth	rx	prescription	wks	weeks
iv	intravenous	tx	treatment	hrs	hours

Table 14: Canonicalisation rules for dates, numbers and units

Class	Before	After	Rule
Date	12/03/2025 · 12-Mar-2025 · 12 March 2025 · 12.03.25	2025-03-12	Four format-specific substitutions to ISO 8601
Patient ID	HX/24/001234 · HX 2024 001234	HX-2024-001234	Unify separators and expand two-digit year
Phone	+91 98765 43210 · 98765-43210	9876543210	Delete non-digits, keep last 10
Number	12,500	12500	Delete thousands separator
Unit	500mg · 500 MG · 0.5ml	500 mg · 0.5 ml	Split from number, lower-case, canonical spelling (mmHg, g/dL, IU kept)
Temperature	98.6 F · 98.6 degrees F · 37.0°C	37.0 °C	Recognise unit words, convert °F to °C (1 d.p.)
Frequency	q8h · b.d.	every 8 hours · twice daily	Pattern and dictionary rule

Complete before-and-after text of record R0106 (prescription).

BEFORE
Rx for HX-2025-086992, aged 17, Lakshmi. amlodipine 75 mg OD for 2 days. Cefixime 0.5ml 1-0-0 x 7 days.
Cap. losartan 5mg BD for the past 14 months. Metformin 40 mg BD for 14 days. Continue therapy for 6 yrs;
call +91 92538 56759 if rash coughs. Avoid pain; treat the fracture and examine the hip. Cap. salbutamol
40 mcg b.d. for 4 days. Continue therapy for 2 yrs; call 9389875069 if cough vomits.

AFTER
prescription for HX-2025-086992 , aged 17 , lakshmi . amlodipine 75 mg once daily for 2 days . cefixime
0.5 ml 1 - 0 - 0 x 7 days . capsule losartan 5 mg twice daily for the past 14 months . metformin 40 mg twice daily for 14 days . continue therapy for 6 years ; call 9253856759 if rash coughs . avoid pain ; treat the fracture and examine the hip . capsule salbutamol 40 mcg twice daily for 4 days . continue therapy for 2 years ; call 9389875069 if cough vomits .

Every drug name is lower-cased, “OD”, “BD” and “b.d.” become “once daily” and “twice daily”, “Cap.” becomes “capsule”, “yrs” becomes “years”, units are split from numbers and the phone number is reduced to ten digits. Punctuation is kept as separate tokens (the spaces before commas are token boundaries, not errors).

4. Irreversible normalisation decisions

Table 15: Information destroyed by normalisation and when the loss is acceptable

Decision	Information destroyed	Acceptable?
Case folding	Distinguishes "US" (ultrasound) from "us", "AS" (aortic stenosis) from "as", "Ca" from "CA".	Acceptable for search (clinicians do not type case) but not for abbreviation resolution; therefore codes and identifiers are protected.
Abbreviation expansion	"OD" becomes "once daily" even where it means right eye; the original spelling is gone.	Acceptable only if the original is kept alongside; wrong for the ophthalmology notes, so I keep the raw text and index both.
Date to ISO	"04/05/2025" is read as day/month; if the writer used month/day the date is wrong and nothing records the guess. Two-digit years assume 20xx.	Acceptable inside one hospital with a known convention, not for imported records.
°F to °C conversion	Rounding to 0.1 °C loses the written precision and the original unit.	Acceptable for search and trends; not for the legal record.
Phone to digits	Spacing, "+91" and bracket style disappear.	Acceptable: identity of the number is preserved.

5. A requirement no regular expression can express

Operation notes sometimes contain nested bracketed asides, for example "Findings: ((Hb 8.2 (low)) with (mild (bilateral)) oedema)". A rule "every opening bracket is closed, in the right order" needs to remember how many brackets are open. A regular expression has only a fixed amount of memory (the finite set of states of its automaton), so for any fixed machine there is a nesting depth it cannot count. Balanced bracket matching therefore needs a stack, i.e. a context-free grammar. I prove this formally in Task 4 with the pumping lemma, and in practice the tokenizer simply emits the brackets as PUNCT tokens and leaves nesting to a later stage.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence	5 Marks

Guiding Enquiry — How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as (Q, Σ, δ, q_0, F), then design DFAs for at least four validators with diagram and table.
- Design an NFA with ε-transitions for an entity with optional parts and trace one accepted and one rejected string, showing live states.
- Convert that NFA to a DFA by subset construction (full table), then minimise and report state counts.
- Apply Thompson's construction to a Task 2 regular expression and derive a regular expression back from an automaton.
- State the pumping lemma and prove a clinical pattern is not regular.

Evidence Expected — Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

Ans:

1. Formal definition and four deterministic validators

A deterministic finite automaton is a five-tuple **(Q, Σ, δ, q_0, F)**: Q a finite set of states, Σ a finite alphabet, δ: Q × Σ → Q the transition function (partial here; a missing entry means the implicit dead state), $q_0 \in Q$ the start state and $F \subseteq Q$ the accepting states. A string is accepted when the run from q_0 ends in F. To keep the tables readable, Σ is a set of character classes: D = digit, U = upper-case letter, and literal characters.

I designed four validators. Each is stricter than its Task 2 regular expression (it checks the whole string, with no context).

DFA-1 ICD-10 code – U D D [. D{1,4} U?] (10 states)

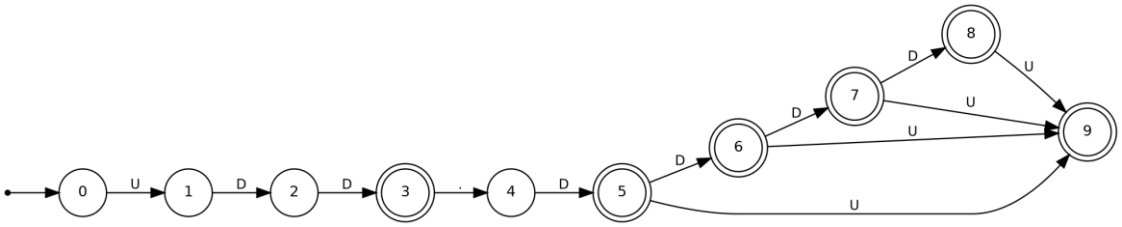


Figure 2: ICD-10 code DFA (U = upper-case letter, D = digit, any missing transition goes to an implicit dead state)

Table 16: Transition table of the ICD-10 code DFA (10 states; → start, * accepting)

State	U	D	.
→0	1	—	—
1	—	2	—
2	—	3	—
*3	—	—	4
4	—	5	—
*5	9	6	—
*6	9	7	—
*7	9	8	—
*8	9	—	—
*9	—	—	—

Accepted: S72.001A ✓, I10 ✓, K35.80 ✓, E11.9 ✓. Rejected: I1 ✓, s72.001 ✓, I10. ✓, 10I ✓.

DFA-2 Blood pressure – D{2,3} / D{2,3} (8 states)

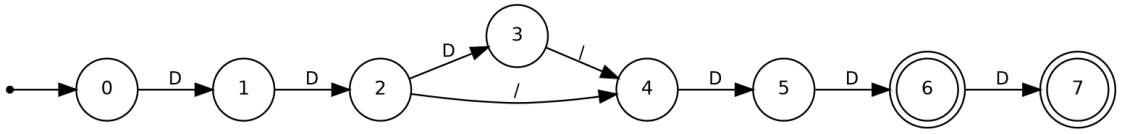


Figure 3: Blood pressure DFA (D = digit, any missing transition goes to an implicit dead state)

Table 17: Transition table of the Blood pressure DFA (8 states; → start, * accepting)

State	D	/
→0	1	—
1	2	—
2	3	4
3	—	4
4	5	—
5	6	—
*6	7	—
*7	—	—

Accepted: 120/80 ✓, 95/60 ✓, 154/98 ✓. Rejected: 1/80 ✓, 120/8 ✓, 1200/80 ✓, 120-80 ✓.

DFA-3 Ward/bed code – W d [A–D] / B d d? (8 states)

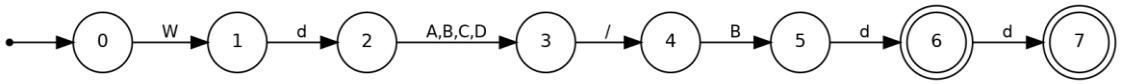


Figure 4: Ward/bed code DFA (d = digit, any missing transition goes to an implicit dead state)

Table 18: Transition table of the Ward/bed code DFA (8 states; → start, * accepting)

State	W	d	A	B	C	D	/
→0	1	—	—	—	—	—	—
1	—	2	—	—	—	—	—
2	—	—	3	3	3	3	—
3	—	—	—	—	—	—	4
4	—	—	—	5	—	—	—

State	W	d	A	B	C	D	/
5	—	6	—	—	—	—	—
*6	—	7	—	—	—	—	—
*7	—	—	—	—	—	—	—

Accepted: W4B/B12 ✓, W2A/B5 ✓. Rejected: W4E/B12 ✓, W4B/B ✓, W4B B12 ✓.

DFA-4 Dose frequency – D-D-D (6 states)

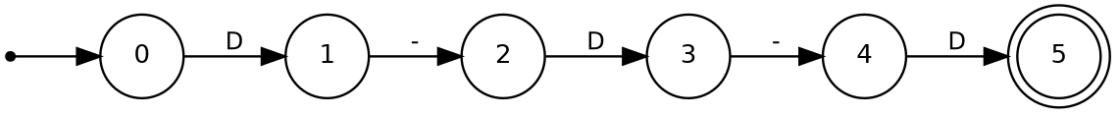


Figure 5: Dose frequency D-D-D DFA (D = digit, any missing transition goes to an implicit dead state)

Table 19: Transition table of the Dose frequency D-D-D DFA (6 states; → start, * accepting)

State	D	-
→0	1	—
1	—	2
2	3	—
3	—	4
4	5	—
*5	—	—

Accepted: 1-0-1 ✓, 0-0-1 ✓. Rejected: 1-0 ✓, 10-1-1 ✓, 1/0/1 ✓.

2. NFA with ε-transitions: body temperature

The temperature format has three optional parts (a third digit, a decimal part, a space and a degree sign) and two alternative final letters: DD D? (. D)? ␣? °? (C|F). Writing the optional parts as ε-moves is natural; a DFA would have to decide at each point which options are still open. The NFA has 11 states (0–10) and five ε-transitions; states 9 and 10 are accepting (C and F).

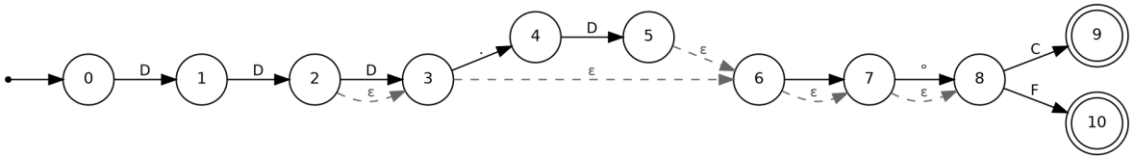


Figure 6: NFA for temperature (dashed edges are ε-transitions)

Table 20: Trace of the accepted string "37.2°C" – set of live states after every step

Step	Symbol class	Live states (after ε-closure)
—	ε-closure of q0	{q0}
read "3"	D	{q1}
read "7"	D	{q2, q3, q6, q7, q8}
read "."	.	{q4}
read "2"	D	{q5, q6, q7, q8}
read "°"	°	{q8}
read "C"	C	{q9}

The final set {q9} contains the accepting state, so "37.2°C" is **accepted**. Note that after the second digit the live set is {q2, q3, q6, q7, q8}: the ε-moves have already skipped the optional third digit, decimal part, space and degree sign, which is exactly the non-determinism.

Table 21: Trace of the rejected string "37.2 K"

Step	Symbol class	Live states (after ε-closure)
—	ε-closure of q0	{q0}
read "3"	D	{q1}
read "7"	D	{q2, q3, q6, q7, q8}
read "."	.	{q4}
read "2"	D	{q5, q6, q7, q8}
read " "		{q7, q8}

Step	Symbol class	Live states (after ε-closure)
read “K”	K	{}

After the space the live set is {q8}; the symbol “K” has no transition from q8, so the set becomes empty and the string is **rejected**.

3. Subset construction and minimisation

Table 22: Complete subset-construction table (A = ε-closure of q0)

DFA state	NFA states	D	␣	.	C	F	°	Accept
A	{q0}	B	∅	∅	∅	∅	∅	
B	{q1}	C	∅	∅	∅	∅	∅	
C	{q2,q3,q6,q7,q8}	D	E	F	G	H	I	
D	{q3,q6,q7,q8}	∅	E	F	G	H	I	
E	{q7,q8}	∅	∅	∅	G	H	I	
F	{q4}	J	∅	∅	∅	∅	∅	
G	{q9}	∅	∅	∅	∅	∅	∅	✓
H	{q10}	∅	∅	∅	∅	∅	∅	✓
I	{q8}	∅	∅	∅	G	H	∅	
J	{q5,q6,q7,q8}	∅	E	∅	G	H	I	

Only 10 of the $2^{11} = 2{,}048$ possible subsets are reachable, so the practical cost of removing non-determinism here is small; the worst case for an n-state NFA is 2^n DFA states, which is the real price of determinism.

Table 23: Minimisation by partition refinement (Moore); ∅ is the dead state

Round	Partition of states
0 (accepting / non-accepting)	{A,B,C,D,E,F,I,J,∅} {G,H}
1	{A,B,F,∅} {C,D,E,I,J} {G,H}
2	{A,∅} {B,F} {C} {D,J} {E} {G,H} {I}
3	{A} {B} {C} {D} {E} {F} {G,H} {I} {J} {∅}
4	{A} {B} {C} {D} {E} {F} {G,H} {I} {J} {∅}

The partition stabilises after round 4: states G (read “C”) and H (read “F”) are both accepting with no outgoing transitions, so they are equivalent and merge into one state. The DFA therefore shrinks from **10 states to 9 states** (the dead state is not counted).

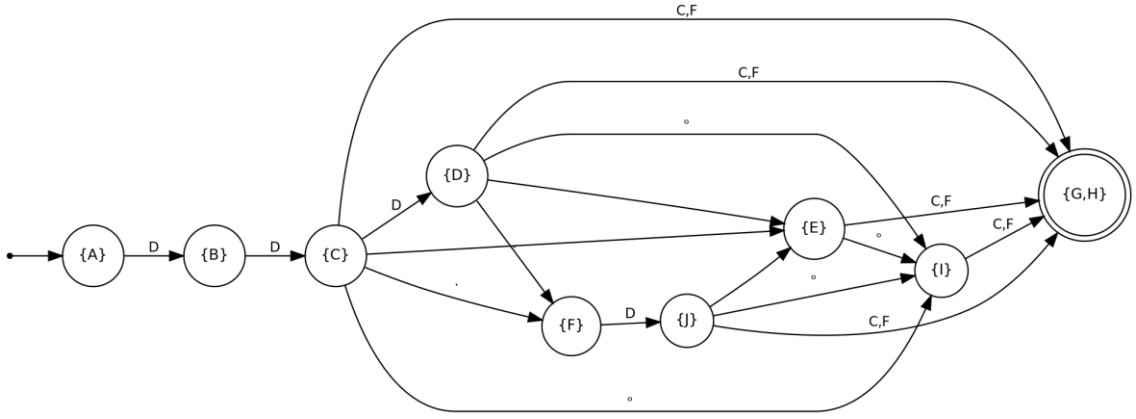


Figure 7: Minimal DFA for temperature (nine states)

Table 24: Transition table of the minimal DFA

State	D	␣	.	C	F	°
→{A}	{B}	—	—	—	—	—
{B}	{C}	—	—	—	—	—
{C}	{D}	{E}	{F}	{G,H}	{G,H}	{I}
{D}	—	{E}	{F}	{G,H}	{G,H}	{I}
{E}	—	—	—	{G,H}	{G,H}	{I}

State	D	␣	.	C	F	°
{F}	{J}	—	—	—	—	—
{G,H	—	—	—	—	—	—
{I}	—	—	—	{G,H}	{G,H}	—
{J}	—	{E}	—	{G,H}	{G,H}	{I}

4. Two-way equivalence with regular expressions

Regular expression → automaton (Thompson)

I applied Thompson's construction to the temperature regular expression DD(D)?(\.D)? ?°?(C|F) (class symbols). Each operator adds fresh start and end states joined by ε-moves; the result has **28 states and 23 ε-transitions**.



Figure 8: Thompson NFA for the temperature expression

Check: I simulated this NFA and Python's re on all 55,987 strings of length ≤ 6 over {1, ., ␣, °, C, F}; there were **0 disagreements**. Examples: 37.2°C ✓, 98 F ✓, 101.4C ✓, 3.5C ✗, 37.2 K ✗.

Automaton → regular expression (state elimination)

From DFA-2 (blood pressure) I added a new start and final state and eliminated states q0…q7 one at a time, combining the paths through each with union and concatenation. The result is ((DD/|DDD/)DD|(DD/|DDD/)DDD), which simplifies (common factor) to **`(DD|DDD)/(DD|DDD)`**: two or three digits, a slash, two or three digits, i.e. the Task 2 blood-pressure expression. I checked the unsimplified expression against the DFA on all 9,841 strings of length ≤ 8 over {1, /, x}: **0 disagreements**. Both directions of Kleene's theorem are therefore demonstrated on my own clinical formats.

5. Non-regularity: the pumping lemma

Pumping lemma. If L is a regular language there is a pumping length $p \geq 1$ such that every string $s \in L$ with $|s| \geq p$ can be written $s = xyz$ with (1) $|y| \geq 1$, (2) $|xy| \leq p$, and (3) $xy^iz \in L$ for every $i \geq 0$.

Claim. The pattern "a note whose brackets are balanced", restricted to the sub-language $L = \{ (^n)^n : n \geq 0 \}$, is not regular. (If the larger bracket language were regular, its intersection with the regular language () would be regular, so L would be too.)

Proof. Suppose L were regular with pumping length p. Take $s = (^p)^p \in L$, $|s| = 2p \geq p$. By (2), xy lies inside the first p characters, so y consists only of opening brackets, and by (1) $y = (^k$ with $k \geq 1$. Pumping up with i = 2 gives $xy^2z = (^{p+k})^p$, which has more opening than closing brackets, so it is not in L. This contradicts (3), hence L is not regular. ■

Relation to Task 3. This is the formal reason that the nested notes of Task 3 cannot be validated by the tokenizer: any automaton with q states loses count beyond nesting depth q. In the pipeline the brackets are emitted as PUNCT tokens and, if nesting matters, a pushdown (context-free) component would be needed after the finite-state layer.

TASK 5 – Inflectional Morphology	3 Marks

Guiding Enquiry — Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation by category preservation, productivity and position.
- List every inflectional paradigm in the corpus and give the full paradigm for at least five lexemes.
- State the orthographic rules precisely enough to implement and give a counter-example on which each over-generates.
- Identify irregular forms and explain why each must be listed.
- Measure the effect of inflection: type-to-lemma ratio and the proportion of query failures explained by inflection alone.

Evidence Expected — Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

Ans:

1. Definitions

Table 25: Key terms with clinical examples

Term	Definition	Example
Morpheme	Smallest meaning-bearing unit	fracture, -s, -ed, nephr-, -ectomy
Stem	The form an affix attaches to (root plus any derivational affixes)	fracture in fractures; infection in infections
Affix	Bound morpheme added to a stem (prefix, suffix)	-ing, re-, -itis
Inflection	Grammatical variant of one lexeme; category unchanged	fracture → fractures, fractured, fracturing
Paradigm	The full set of inflected forms of one lexeme	bleed: bleed, bleeds, bled, bled, bleeding

Table 26: Inflection versus derivation

Criterion	Inflection	Derivation
Category	Preserved: fracture (V) → fractured (V); fracture (N) → fractures (N)	Often changed: infect (V) → infection (N) → infectious (A)
Productivity	Fully productive: every regular noun takes -s	Partial: nephr-ectomy exists but "*cardi-ectomy" is not licensed
Position	Outermost: nephrectomy-ies, re-infect-ion-s	Inside inflection: nephr-ectomy-ies
Meaning	Regular and predictable (plural, past)	Can drift (non-compositional, Task 6)
Obligatoriness	Required by syntax (agreement, tense)	Optional, a lexical choice

2. Inflectional paradigms in the corpus

Paradigm	Features	Corpus examples
Noun number	SG, PL	wound/wounds, abscess/abscesses, vertebra/vertebrae
Verb tense and aspect	BASE, PAST, PSP (past participle), PRP (-ing)	observe/observed/observing, give/gave/given
Subject agreement	3SG	monitors, carries, dresses
Adjective comparison	POS, CMP, SUP	high/higher/highest, big/bigger/biggest, bad/worse/worst

Possessives and plural possessives do not occur in the records. Verb forms are ambiguous between PAST and PSP for every regular verb (fractured), which is a property of English, not of my analyser.

Table 27: Full paradigms of eight clinical lexemes

Lexeme	POS	Cells	Forms
fracture	V	BASE · 3SG · PAST · PSP · PRP	fracture · fractures · fractured · fractured · fracturing
infection	N	SG · PL	infection · infections
bleed	V	BASE · 3SG · PAST · PSP · PRP	bleed · bleeds · bled · bled · bleeding
vertebra	N	SG · PL	vertebra · vertebrae
stomach	N	SG · PL	stomach · stomachs
carry	V	BASE · 3SG · PAST · PSP · PRP	carry · carries · carried · carried · carrying
admit	V	BASE · 3SG · PAST · PSP · PRP	admit · admits · admitted · admitted · admitting
heavy	A	POS · CMP · SUP	heavy · heavier · heaviest

3. Inflectional affix inventory (24 realisations)

#	Affix	Feature	Example	#	Affix	Feature	Example
1	-s	N PL	wounds	13	-ies (y→ie+s)	V 3SG	carries
2	-es	N PL	rashes	14	-ed	V PAST/PSP	observed
3	-ies	N PL	allergies	15	-ied	V PAST/PSP	carried

#	Affix	Feature	Example	#	Affix	Feature	Example
4	-ves	N PL	leaves	16	-ing	V PRP	monitoring
5	-a (-um)	N PL	bacteria	17	-ying (ie→y)	V PRP	lying
6	-ae	N PL	vertebrae	18	-en	V PSP	taken, swollen
7	-i (-us)	N PL	bronchi	19	-t	V PAST/PSP	kept, felt
8	-ices (-ix)	N PL	appendices	20	-er	A CMP	higher
9	-es (-is)	N PL	diagnoses	21	-ier	A CMP	heavier
10	-ren, -en, mutation	N PL	children, teeth	22	-est	A SUP	deepest
11	-s	V 3SG	monitors	23	-iest	A SUP	heaviest
12	-es	V 3SG	dresses	24	suppletion	A CMP/SUP	worse, best

4. Orthographic rules and where each over-generates

Table 28: Rules stated precisely and a counter-example for each

Rule (implemented as in Task 8)	Precise statement	Over-generation (rule output vs correct)
Consonant doubling (R1)	Before a vowel-initial suffix, double the final consonant of a stem that ends consonant–vowel–consonant (not w, x, y) and is stressed on its last syllable	visit → visitted (correct: visited); offer → offerred (correct: offered); target → targetted (correct: targeted); benefit → benefitted (correct: benefited); focus → focussed (correct: focused). The spelling alone does not show stress, so the lexicon must flag the stem (D).
e-deletion (R4)	Delete a stem-final e before a vowel-initial suffix, unless it follows e, o or y	see → seing (correct: seeing); dye → dying (correct: dyeing). The context "not after e/o/y" repairs "see", but not "dye".
Sibilant e-insertion (R6)	Insert e between a stem ending in s, x, z, ch or sh and the suffix -s	stomach → stomaches (correct: stomachs). Pronunciation (ch = /k/) decides, not spelling.
y-rules (R5)	After a consonant, y → ie before -s and y → i before -ed, -er, -est, -ness; y stays before -ing	kidney → kidneies (correct: kidneys); day → daies (correct: days). The "consonant" context repairs these, but "dry+ness" → driness (correct: dryness) still needs a listed exception.

5. Irregular forms

I list 34 irregular nouns, 34 irregular verbs and 4 irregular adjectives in the lexicon (the full lists are in the submitted lexicon file). Each must be listed because its stem-to-form relation is not a general spelling rule: ablaut (give → gave, take → took), suppletion (bad → worse, go → went), zero marking (cut, put, hurt), and Latin/Greek plurals (vertebra → vertebrae, bronchus → bronchi, appendix → appendices, diagnosis → diagnoses, datum → data). A rule such as "-us → -i" would be wrong for "virus" and "bus", and no rule predicts "bled" from "bleed".

Table 29: Most frequent irregular forms in the corpus

Form	Lemma	Tokens	Form	Lemma	Tokens
bled	bleed	134	swelled	swell	117
cut	cut	44	laid	lay	44
found	find	44	felt	feel	42
left	leave	39	children	child	36
sat	sit	36	fistulae	fistula	36

6. Measuring the effect of inflection

Table 30: Quantified effect of inflection

Measure	Value	Meaning
Word types analysed	507	Distinct surface forms the transducer analyses
Lemmas	344	Distinct lemmas behind those forms

Measure	Value	Meaning
Type-to-lemma ratio	1.47	Each lemma appears on average as 1.47 surface forms; every query term has to be matched against about one and a half spellings
Relevant (query, record) pairs missed by exact matching	1,964	Records that contain the concept but not the typed word
…recovered by inflectional lemmatisation alone	1,670 (85.0%)	Share of failures explained by inflection alone
…still missed after lemmatisation	294	Derivational relatives (operate → operation), explained in Task 6

Inflection alone therefore explains 85.0% of the missed relevant records, which is why even the cheapest morphological component (Porter or the lemmatiser) lifts macro recall from 56.5% to above 92.0%.

TASK 6 – Derivational Morphology	4 Marks

Guiding Enquiry — How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes with their category change.
- Build the derivational family of at least six roots, showing attachment order and two impossible orders.
- Explain semantic non-compositionality and what it implies for aggressive stemming.
- Compare the productivity of the affixes empirically (types and tokens) and classify each as productive or fossilised.
- Decide whether derivational relatives should be conflated, citing two helpful and two misleading cases.

Evidence Expected — A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

Ans:

1. Derivation and the affix inventory

Derivation forms a new lexeme from an existing stem or bound root by adding an affix. Unlike inflection it usually changes the category (infect V → infection N), is lexically restricted, can change the meaning unpredictably and attaches inside inflection. The inventory below has **25 suffixes and 16 prefixes (41 affixes)**; all are attested in the lexicon, and the counts in Section 4 show which occur in the corpus.

Table 31: Derivational affixes (41), category change, meaning and example

Affix	Type	Category change	Meaning	Example
-itis	suffix (bound root)	root→N	inflammation of	gastritis
-ectomy	suffix (bound root)	root→N	surgical removal of	nephrectomy
-otomy	suffix (bound root)	root→N	cutting into	nephrotomy
-ostomy	suffix (bound root)	root→N	creating an opening into	colostomy
-oscopy	suffix (bound root)	root→N	looking into	gastroscopy
-ology	suffix (bound root)	root→N	study of	nephrology
-ologist	suffix (bound root)	root→N	specialist in	neurologist
-osis	suffix (bound root)	root→N	abnormal condition	nephrosis
-aemia	suffix (bound root)	root→N	blood condition	hypoglycaemia
-algia	suffix (bound root)	root→N	pain in	neuralgia
-opathy	suffix (bound root)	root→N	disease of	neuropathy
-oplasty	suffix (bound root)	root→N	surgical repair of	arthroplasty
-ation	suffix	V→N	act/result of	examination
-ion	suffix	V→N	act/result of	operation
-able	suffix	V→A	can be …-ed	treatable
-ive	suffix	V→A	tending to	operative

Affix	Type	Category change	Meaning	Example
-ment	suffix	V→N	result of	treatment
-ness	suffix	A→N	state of	mildness
-ity	suffix	A→N	quality of	severity
-al	suffix	N→A	relating to	medicinal
-ic	suffix	N→A	relating to	organic
-ist	suffix	N→N	practitioner of	therapist
-ly	suffix	A→ADV	in a … manner	mildly
-less	suffix	N→A	without	painless
-ful	suffix	N→A	full of	painful
hyper-	prefix	A/N→same	above normal	hypertension
hypo-	prefix	A/N→same	below normal	hypotension
re-	prefix	V/N→same	again	readmit
un-	prefix	A/V→same	not	unstable
post-	prefix	A→A	after	post-operative
pre-	prefix	A→A	before	pre-operative
peri-	prefix	N/A→same	around	peri-operative
intra-	prefix	A→A	within	intra-operative
anti-	prefix	N/A→same	against	anti-inflammatory
dys-	prefix	N→N	abnormal	dysfunction
brady-	prefix	ROOT→ROOT	slow	bradycardia
tachy-	prefix	ROOT→ROOT	fast	tachycardia
poly-	prefix	N→N	many	polytherapy
non-	prefix	A→A	not	non-invasive
inter-	prefix	A→A	between	intercostal
sub-	prefix	N/A→same	under	sublingual

2. Derivational families and attachment order

Table 32: Derivational families of six clinical roots (all licensed by the lexicon)

Root	Family (root + suffix)
nephr-	nephritis, nephrectomy, nephrotomy, nephrostomy, nephrology, nephrologist, nephrosis, nephropathy
gastr-	gastritis, gastrectomy, gastrotomy, gastrostomy, gastroscopy, gastrology, gastropathy, gastralgia
hepat-	hepatitis, hepatectomy, hepatology, hepatologist, hepatosis, hepatopathy, hepatotomy
arthr-	arthritis, arthroscopy, arthrotomy, arthroplasty, arthralgia, arthrosis, arthrectomy, arthropathy
cyst-	cystitis, cystectomy, cystotomy, cystostomy, cystoscopy
neur-	neuritis, neurology, neurologist, neurosis, neuralgia, neuropathy, neurectomy, neurotomy

The attachment order is fixed: **[prefix] – root or stem – derivational suffix(es) – inflectional suffix**, as in re-|infect|-ion|-s (reinfections) or nephr|-ectomy|-ies. Two orders are impossible, and the analyser indeed returns no analysis for them:

Impossible order	Why impossible	Analyser output
nephrsectomy (inflection inside derivation)	Inflection must be outermost: plural goes after -ectomy.	no analysis
ectomynephr (suffix before roo	A suffix cannot precede the root it attaches to.	no analysis
infectionre (prefix after suffi	Prefixes attach to the left edge only.	no analysis

3. Semantic non-compositionality

Table 33: Derived words whose meaning cannot be predicted from the parts

Word	Predicted from parts	Actual meaning
nephrosis	nephr- + -osis: "abnormal condition of the kidney"	A specific degenerative, non-inflammatory kidney disease; contrasts with nephritis (inflammatory) although both are "kidney conditions".
intern / internal	internal = "of an intern"?	Unrelated: a junior doctor versus "inside the body"; Porter maps both to intern.
medication / medical	medic- + -ation / -al	The drug given versus anything relating to medicine; Porter maps both to medic.
general / generation	gener- + -al / -ation	General anaesthesia versus a drug generation; both become gener under Porter.
dressing	dress + -ing	A wound covering, not the act of dressing a patient.
hypertension	hyper- + tension: "above normal pressure"	Compositional in meaning, but opposite of hypotension; a stemmer that stripped prefixes would merge the two.

For a search system this means that aggressive stemming trades precision for recall in a way that cannot be undone by weighting alone: after stemming, "medical" and "medication" are the same index term, so no scoring function can tell them apart. Derivation must therefore be conflated selectively, not by default.

4. Productivity of the affixes

For each affix I counted its **types** (distinct word forms) and **tokens** in the corpus, the **hapax legomena** (types occurring once), and the **licensed bases** (how many different stems or roots the lexicon lets it attach to). Because the corpus is template-generated, every type is repeated and there are no hapaxes, so Baayen's hapax-based productivity cannot discriminate; I therefore classify by the number of licensed bases, using corpus types and tokens as evidence that the affix is attested: **productive** = 5 or more bases, **limited** = 2–4, **fossilised** = 1 (attested only in one frozen word).

Table 34: Types, tokens and productivity of every affix

Affix	Types	Tokens	Hapax	Licensed bases	Class
-itis	3	53	0	23	productive
-ectomy	8	81	0	18	productive
-otomy	0	0	0	16	productive
-ostomy	2	19	0	8	productive
-oscopy	2	20	0	8	productive
-ology	1	12	0	8	productive
-ologist	0	0	0	7	productive
-osis	0	0	0	7	productive
-aemia	4	76	0	3	limited
-algia	0	0	0	4	limited
-opathy	2	23	0	11	productive
-oplasty	0	0	0	4	limited
-ation	4	99	0	7	productive
-ion	19	679	0	20	productive
-able	0	0	0	4	limited
-ive	5	183	0	1	fossilised
-ment	2	15	0	4	limited
-ness	0	0	0	5	productive
-ity	2	39	0	2	limited
-al	0	0	0	2	limited
-ic	0	0	0	2	limited
-ist	1	72	0	1	fossilised
-ly	1	17	0	7	productive

Affix	Types	Tokens	Hapax	Licensed bases	Class
-less	1	20	0	2	limited
-ful	1	20	0	1	fossilised
hyper-	1	22	0	1	fossilised
hypo-	1	16	0	1	fossilised
re-	0	0	0	5	productive
un-	1	19	0	2	limited
post-	2	134	0	1	fossilised
pre-	2	33	0	2	limited
peri-	0	0	0	1	fossilised
intra-	2	39	0	2	limited
anti-	1	20	0	1	fossilised
dys-	0	0	0	1	fossilised
brady-	1	17	0	1	fossilised
tachy-	1	19	0	1	fossilised
poly-	0	0	0	1	fossilised
non-	0	0	0	1	fossilised
inter-	0	0	0	1	fossilised
sub-	1	20	0	1	fossilised

-ion (19 types, 679 tokens), -ation, -ectomy, -itis and -opathy are productive suffixes with real corpus support; -ive, -ist, -ful and most prefixes (hyper-, hypo-, post-, anti-, brady-, tachy-, sub-) are attested through a single stem and behave as fossilised in my lexicon. Several licensed affixes (-otomy, -ologist, -osis, -algia, -oplasty, -able, -ness, -al, -ic and the prefixes re-, peri-, dys-, poly-, non-, inter-) never occur in the corpus, a reminder that productivity in the lexicon and productivity in text are different things.

5. Should derivational relatives be conflated for search?

Table 35: Cases where conflation helps and where it misleads (measured)

Case	Effect	Evidence
Helps: operate ~ operation ~ operative (query "operate")	A surgeon searching "operate" must find "operation" and "post-operative" notes.	Recall for "operate": 43% with inflection only → 100% with whitelist conflation
Helps: treat ~ treatment, examine ~ examination	Clinicians use the verb and the noun interchangeably.	Recall for "examine": 76% → 100%; "treat": 90% → 100%
Misleads: medical ~ medication	A search for "medical" returns drug-administration notes.	Porter retrieves 15 wrong records for "medical"
Misleads: general ~ generation, organ ~ organization, intern ~ internal, university ~ universal	Unrelated concepts share a stem.	Porter false positives: general 65, generation 78, organ 81, intern 66, university 91 (precision 0.49–0.67)

Position. Conflate inflection always; conflate derivation only through a short **whitelist of semantically transparent suffixes (-ion, -ation, -ment, -ive, -able)** on bases that the lexicon licenses; never conflate across prefixes (hyperglycaemia and hypoglycaemia are opposites) or across the Greek combining suffixes (-itis, -ectomy and -osis name different conditions and procedures: nephritis ≠ nephrectomy). With this policy the analyser reaches recall 100.0% at precision 98.6%, whereas Porter loses precision on every collision above.

TASK 7 – Finite-State Morphological Parsing	5 Marks

Guiding Enquiry — How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define an FST as a two-tape automaton and explain the lexical and surface tapes with a worked example.
- Build the lexicon as an FST organised into continuation classes covering at least three parts of speech and the affixes of Tasks 5 and 6.
- Hand-trace at least five surface forms showing state sequence, symbols consumed and emitted, and final

analysis.

- Run the same transducer in the generation direction and show that parsing and generation use one machine.
- Handle ambiguity: give two ambiguous forms, show all analyses and state how a downstream component chooses.

Evidence Expected — A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

Ans:

1. The transducer and its tapes

A **finite-state transducer** is a finite automaton whose arcs carry a pair of symbols x:y, so it reads one tape and writes another, i.e. it recognises a relation between two strings. Formally $T = (Q, \Sigma_lex, \Sigma_int, \delta, q_0, F)$ with $\delta \subseteq Q \times (\Sigma_lex \cup \{\varepsilon\}) \times (\Sigma_int \cup \{\varepsilon\}) \times Q$. My lexicon transducer relates the **lexical tape** (stem plus feature symbols, e.g. fracture +N +PL) to an **intermediate tape** with explicit morpheme and word boundaries (fracture^s#); the spelling-rule cascade of Task 8 then maps the intermediate tape to the **surface tape** (fractures). Because every arc is a pair, the same arcs can be read left-to-right (generation: lexical → intermediate) or right-to-left (parsing: intermediate → lexical).

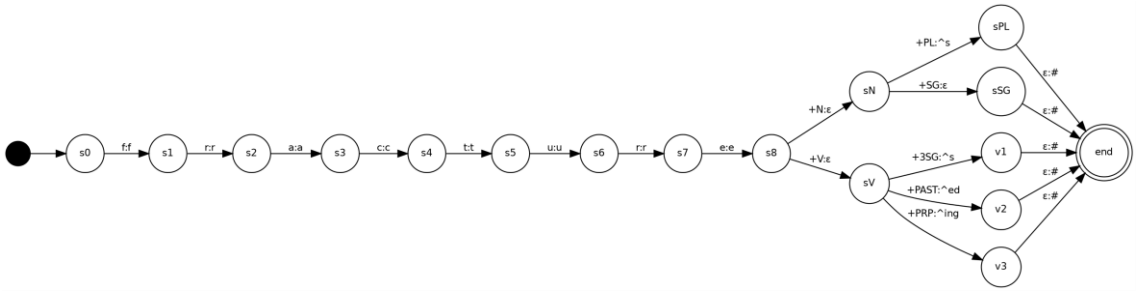


Figure 9: Fragment of the lexicon FST for the stem "fracture" (arc label = lexical symbol : intermediate symbol)

Worked example: parsing starts from the surface form "fractures"; the rules (inverted) give the intermediate string fracture^s#; the transducer reads it symbol by symbol, writing f r a c t u r e for the letters, then writes the features +N +PL on ε-input arcs while reading ^s, and finishes in an accepting state at #. The same path read the other way turns fracture +N +PL into fracture^s#.

2. Lexicon as an FST with continuation classes

Table 36: Continuation classes of the lexicon FST

Class	Stems (examples)	Continues to	Arcs written
N-REG (≈ 250 noun stems)	wound, fracture, kidney, abscess	N-NUM	+N, then +SG:ε or +PL:^s
N-IRR (31)	vertebra, bacterium, diagnosis	final (irregular plural spelled in the path)	+N +PL : listed form
V-REG (≈ 130 verb stems)	observe, admit@, carry, panic$	V-INF	+V then +BASE:ε, +3SG:^s, +PAST:^ed, +PSP:^ed, +PRP:^ing (markers @ and $ record doubling and k-insertion)
V-IRR (36)	bleed, give, take, be	final	listed PAST, PSP, 3SG, PRP
A-GRAD (≈ 60)	high, big@, heavy	A-DEG	+A then +POS:ε, +CMP:^er, +SUP:^est
A-FIXED, A-IRR	renal, acute; good, bad	A-POS / final	+A +POS (+CMP, +SUP listed for irregulars)
DER-SUFFIX (25)	^ION, ^ATION, ^ITIS, ^ECTOMY, …	N-NUM or A-POS	derivational tag then category and number
PREFIX (16)	PRE+hyper, PRE+post, …	stem classes	PRE+x on ε-input arc before the stem
ROOT (33 bound)	nephr, gastr, hepat	DER-SUFFIX only (licensed pairs)	+ROOT then suffix tag

The compiled transducer has **14,034 states and 14,033 arcs** for 2,005 lexical entries (507 stems expanded by their inflections, 168 licensed derivations and prefixed words). It covers three parts of speech (noun, verb, adjective) plus derived adverbs and the inflectional and derivational affixes of Tasks 5 and 6.

3. Hand-traces of five surface forms

Each trace is the actual path found by the program in the parsing direction. State numbers are renamed q0, q1, … in order of visit (the real state ids in the 14,000-state machine are arbitrary). "read" is the symbol consumed from the intermediate tape (ε = nothing consumed) and "write" the symbol emitted on the lexical tape.

fractures → fracture +N +PL | fracture +V +3SG

Rules (inverse): intermediate form fracture^s#.

```
state  q0  q1  q2  q3  q4  q5  q6  q7  q8  q9  q10  q11  q12
read   f   r   a   c   t   u   r   e   ε   ε   ^    s    #
write  f   r   a   c   t   u   r   e   +N  +PL ε    ε    ε

state  q13
read   ·
write  ·
```

admitted → admit +V +PAST | admit +V +PSP

Rules (inverse): intermediate form admit@^ed#; rewriting applied: R1 consonant doubling.

```
state  q0  q1  q2  q3  q4  q5  q6  q7  q8  q9  q10  q11  q12
read   a   d   m   i   t   ε   ε   @   ^   e   d    #    ·
write  a   d   m   i   t   +V  +PAST ε   ε   ε   ε    ε    ·
```

carries → carry +V +3SG

Rules (inverse): intermediate form carry^s#; rewriting applied: R5a y→ie.

```
state  q0  q1  q2  q3  q4  q5  q6  q7  q8  q9  q10
read   c   a   r   r   y   ε   ε   ^   s   #   ·
write  c   a   r   r   y   +V  +3SG ε   ε   ε   ·
```

vertebrae → vertebra +N +PL

Rules (inverse): intermediate form vertebrae#.

```
state  q0  q1  q2  q3  q4  q5  q6  q7  q8  q9  q10  q11  q12
read   v   e   r   t   e   b   r   a   ε   ε   e    #    ·
write  v   e   r   t   e   b   r   a   +N  +PL ε    ε    ·
```

nephrectomies → nephr +ROOT ^ECTOMY +N +PL

Rules (inverse): intermediate form nephrectomy^s#; rewriting applied: R5a y→ie.

```
state  q0  q1  q2  q3  q4  q5  q6  q7  q8  q9  q10  q11  q12
read   n   e   p   h   r   ε   ε   ε   ε   e   c    t    o
write  n   e   p   h   r   +ROOT ^ECTOMY+N +PL ε    ε    ε    ε

state  q13  q14  q15  q16  q17  q18
read   m    y    ^    s    #    ·
write  ε    ε    ε    ε    ε    ·
```

In every trace the stem letters are copied (x:x), the boundary symbols ^ and # are consumed without output, and the feature symbols are written on ε-input arcs. For "vertebrae" the path leaves the regular class: the irregular plural is spelled in its own path (e read while +PL is written), so no rule is involved. For "nephrectomies" the lexical output nephr +ROOT ^ECTOMY +N +PL shows both the bound root and the derivational suffix, and the y → ie rule of Task 8 supplied the "ies".

4. Generation with the same machine

Table 37: Generation: lexical input to surface output

Lexical input	Generated surface form	Rules used
fracture +N +PL	fractures	cleanup
give +V +PSP	given	none (irregular path)
carry +V +PAST	carried	R5b y→i
admit +V +PRP	admitting	R1 doubling
bad +A +CMP	worse	none (suppletive path)

Lexical input	Generated surface form	Rules used
nephr +ROOT ^ECTOMY +N +PL	nephrectomies	R5a y→ie
operate +V ^ION +N +PL	operations	R4 e-deletion

One machine, not two. In the program the FST object has a single arc table; run(tape, "parse") reads the second component of each arc and writes the first, run(tape, "gen") does the opposite. There is no second automaton and no separate generation table. The same compiled arcs that traced “fractures” also generate “fractures” from fracture +N +PL, and the generated forms above were obtained by running the parse-direction path backwards. Only the direction of reading differs.

5. Ambiguity

Table 38: Surface forms with more than one analysis (all returned by the transducer)

Surface form	All analyses returned	Downstream choice
leaves	leaf +N +PL | leave +V +3SG	A part-of-speech tagger: “the leaves were …” (noun) vs “he leaves …” (verb).
saw	saw +N +SG | see +V +PAST	Tagger or surrounding words (“the saw” / “he saw”).
left	leave +V +PAST | leave +V +PSP | left +A +POS	Context: “left ankle” (adjective) vs “left the ward” (verb).
fractured	fracture +V +PAST | fracture +V +PSP	Auxiliary: “was fractured” (PSP) vs “fractured the …” (PAST). For retrieval the lemma is identical, so no choice is needed.
diagnoses	diagnosis +N +PL | diagnose +V +3SG	Number agreement or tagger; retrieval can keep both lemmas.

The transducer is deliberately non-committal: it returns **every** analysis its lexicon licenses, because choosing needs context that a finite-state lexicon does not have. A downstream component would pick the analysis with a part-of-speech tagger (an HMM or maximum-entropy model trained on annotated clinical text) or leave it ambiguous. For search I index all lemmas of an ambiguous token. This costs some precision (the verb “swell” also matches the noun “swelling”, precision 0.45 for that query) but never loses a relevant record.

TASK 8 – Orthographic Rules and Two-Level Morphology	4 Marks

Guiding Enquiry — How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, with at least three examples.
- Write at least five orthographic rules in rewrite notation with precise contexts.
- Implement each rule as a transducer, compose with the lexicon FST, draw the cascade and state order dependencies.
- Contrast the sequential cascade with the parallel two-level formulation.
- Test on at least twenty forms exercising every rule, tabulate successes and failures and account for each failure.

Evidence Expected — Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

Ans:

1. Why the lexicon transducer alone is not enough

The lexicon transducer only concatenates morphemes: its output is the intermediate string, not a spelling. Plain concatenation gives wrong words wherever the boundary changes the letters:

Lexical form	Concatenation	Correct surface	Phenomenon
fracture +V +PRP	fractureing	fracturing	final e drops before a vowel
admit +V +PAST	admited	admitted	final consonant doubles
carry +V +3SG	carrys	carries	y becomes ie
dress +V +3SG	dresss	dresses	e inserts after a sibilant
panic +V +PAST	panicd/panicked?	panicked	k inserts after c

Listing every surface form instead would multiply the lexicon (each of the 507 stems has up to five forms) and would not generalise to new stems. Rules keep the lexicon small and apply to new stems automatically.

2. Orthographic rules in rewrite notation

Notation: a → b / c __ d means "rewrite a as b between left context c and right context d". ^ is the morpheme boundary, # the word end, @ and $ are lexical markers for "double this consonant" and "insert k".

Table 39: The seven rules of the cascade (five orthographic rules plus ie→y and cleanup)

Rule	Rewrite notation	Regular expression implemented	Example
R1 consonant doubling	C → CC / C@ _ ^ V	([^aeiou])@\^(?=[aeiou]) → \1\1^	admit@^ed → admitted
R2 k-insertion	c → ck / _ $^ {e,i}	c\$\^(?=[ei]) → ck^	panic$^ed → panicked
R3 ie→y	ie → y / _ ^ ing	ie\^(?=ing) → y^	lie^ing → lying
R4 e-deletion	e → ε / [^eoy] _ ^ V	(?<![eoy])e\^(?=[aeiou]) → ^	fracture^ing → fracturing
R5a y→ie	y → ie / C _ ^ s	([^aeiou])y\^(?=s) → \1ie^	carry^s → carries
R5b y→i	y → i / C _ ^ {e,ness,ly,…}	([^aeiou])y\^(?=(?:e|ness|ly|ful|ment|able)\b|e) → \1i^	heavy^er → heavier
R6 e-insertion	ε → e / {s,x,z,ch,sh} ^ _ s	(s|x|z|ch|sh)\^(?=s) → \1e^	dress^s → dresses
R7 cleanup	boundary and marker deletion	[\^#@$%] → ε	fracture^s# → fractures

3. Rules as transducers, composition and order

Each rule is a regular relation (Kaplan and Kay): it reads the intermediate tape left to right and rewrites at most one boundary context with bounded look-behind and look-ahead, which a finite-state transducer can do. In the program each rule is implemented as a left-to-right regex rewrite, which is the same relation; the cascade applies them in sequence. Composing the lexicon FST with R1 ∘ R2 ∘ … ∘ R7 gives one transducer from the lexical tape to the surface tape. Figure below shows the cascade.

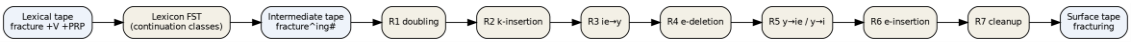


Figure 10: Lexicon FST composed with the rule cascade (lexical tape → surface tape)

Order dependencies. I tested the order empirically: for each pair of rules I swapped their positions and re-ran the cascade on 18 intermediate forms. Only the pairs below change any output; all other pairs commute because their contexts never overlap.

Table 40: Order dependencies found by swapping rules

Rules swapped	Forms changed	Example (intermediate → correct vs swapped)	Reason
R1 consonant doubling ↔ R4 e-deletion	1	lie^ing# → lying vs liing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.
R2 k-insertion ↔ R4 e-deletion	1	lie^ing# → lying vs liing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.
R3 ie→y ↔ R4 e-deletion	1	lie^ing# → lying vs liing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.
R3 ie→y ↔ R5a y→ie	1	lie^ing# → lying vs liing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.

Rules swapped	Forms changed	Example (intermediate → correct vs swapped)	Reason
R3 ie→y ↔ R5b y→i	1	lie^ing# → lying vs liing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.
R3 ie→y ↔ R6 e-insertion	1	lie^ing# → lying vs liing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.
R7 cleanup ↔ (moved to first)	16	fracture^ing# → fracturing vs fractureing	R3 must precede R4: otherwise e-deletion turns lie^ing into li^ing → liing; R3 first gives lying.

4. Cascade versus two-level (Koskenniemi) morphology

	Sequential cascade (this work)	Parallel two-level (Koskenniemi)
Idea	Rules applied one after another; each rewrites the output of the previous one through intermediate tapes.	Rules are constraints on lexical:surface symbol pairs; all rules run in parallel on a single pair of tapes (e.g. e:0 ⇔ C __ +:0 V).
Practical advantage	Easy to write and read (familiar rewrite notation); each rule can be tested alone; intermediate forms are inspectable.	No intermediate tapes and no rule ordering problems (rules commute); naturally bidirectional; faster to run and compile into a single machine.
Practical cost	Order matters (R3 before R4, R7 last) and mistakes interact; intermediate tapes can grow.	Constraints can be hard to formulate and conflicts between rules must be resolved by the grammar writer; harder to debug.

5. Tests on 22 forms

Table 41: Twenty-two forms exercising every rule: expected, pure-rule output and final output

Lemma	Features	Expected	Rules only (no exception flag)	Final (with lexical flags)	Rule exercised	OK
fracture	+V +PRP	fracturing	fracturing	fracturing	R4	✓
discharge	+V +PAST	discharged	discharged	discharged	R4	✓
admit	+V +PRP	admitting	admitting	admitting	R1	✓
stop	+V +PAST	stopped	stopped	stopped	R1	✓
refer	+V +PAST	referred	referred	referred	R1	✓
carry	+V +3SG	carries	carries	carries	R5a	✓
carry	+V +PAST	carried	carried	carried	R5b	✓
heavy	+A +CMP	heavier	heavier	heavier	R5b	✓
allergy	+N +PL	allergies	allergies	allergies	R5a	✓
dress	+V +3SG	dresses	dresses	dresses	R6	✓
rash	+N +PL	rashes	rashes	rashes	R6	✓
fix	+V +3SG	fixes	fixes	fixes	R6	✓
abscess	+N +PL	abscesses	abscesses	abscesses	R6	✓
panic	+V +PAST	panicked	panicked	panicked	R2	✓
mimic	+V +PRP	mimicking	mimicking	mimicking	R2	✓
lie	+V +PRP	lying	lying	lying	R3	✓
see	+V +PRP	seeing	seeing	seeing	R4-block	✓

Lemma	Features	Expected	Rules only (no exception flag)	Final (with lexical flags)	Rule exercised	OK
big	+A +CMP	bigger	bigger	bigger	R1	✓
kidney	+N +PL	kidneys	kidneys	kidneys	none	✓
visit	+V +PAST	visited	visited	visited	R1-block	✓
stomach	+N +PL	stomachs	stomaches	stomachs	R6-block	✓
biopsy	+N +PL	biopsies	biopsies	biopsies	R5a	✓

Result. With the rules alone 21 of 22 forms are correct; with the lexical exception flag 22 of 22 are correct. The one failure of the bare rules is stomach → stomaches: rule R6 fires on every stem ending in "ch", but here ch is pronounced /k/ and English writes "stomachs". No spelling rule can know this, so the lexicon carries a flag (%) on the stem that blocks R6. The blocked cases "see+ing" (R4 not applied after ee) and "visit+ed" (R1 not applied because visit is not flagged for doubling) pass because the context* or the lexical flag prevents the over-generation described in Task 5. Every failure is therefore accounted for: it is a lexically conditioned exception, not a bug in a rule.

TASK 9 – The Porter Stemmer	5 Marks

Guiding Enquiry — What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: consonant and vowel, the measure m, the conditions v, d and o, with calculations for at least six clinical words.
- Work through all five steps by hand for at least six words.
- Contrast stemming with lemmatisation using the Task 7 transducer on the 200 annotated forms.
- Quantify over-stemming and under-stemming with clinical examples and report rates.
- Measure the retrieval effect: no stemming, Porter and full morphological analysis (precision, recall, F1).

Evidence Expected — Correct hand-traces, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

Ans:

1. The algorithm

A **vowel** is a, e, i, o, u, or y when it follows a consonant; every other letter is a **consonant** (y at the start or after a vowel is a consonant). Writing a word as $[C](VC)^m[V]$, the **measure m** is the number of VC sequences. Conditions on a stem: *\v* **contains a vowel;** \d ends in a double consonant; \o** ends consonant–vowel–consonant where the last consonant is not w, x or y. The algorithm applies five ordered steps (1a plural, 1b/1c -ed/-ing and y, 2 and 3 derivational suffixes, 4 suffix deletion when m > 1, 5 final -e and -ll), taking the longest matching suffix in each step.

Table 42: Computation of the CV form, m and the conditions for eight clinical words

Word	CV form (collapsed)	m	v	*d	*o
tree	cv	0	yes	no	no
trouble	cvcv	1	yes	no	no
oats	vc	1	yes	no	no
fracture	cvcvcv	2	yes	no	no
nephrectomy	cvcvcvcv	3	yes	no	no
hop	cvc	1	yes	no	yes
fill	cvc	1	yes	yes	no
infect	vcvc	2	yes	no	no

Worked: fracture = f-r (C) a (V) c-t (C) u (V) r (C) e (V) → C V C V C V → [C] VC VC [V] → m = 2. nephrectomy = C V C V C V C V → m = 3. hop = CVC gives m = 1 and satisfies o; fill* ends in a double consonant (*d) with m = 1.

2. Step-by-step traces

Table 43: Rules that fire and the form after each step (eight words)

Word	m	Rule that fires → intermediate form	Stem
fractures	3	Step 1a S → ε: fractures → fracture; Step 5a E → ε: fracture → fractur	fractur
fracturing	3	Step 1b (v) ING → ε: fracturing → fractur	fractur
hopping	2	Step 1b (v) ING → ε: hopping → hopp; Step 1b (d & ¬(L,S,Z)) → single letter: hopp → hop	hop
infection	3	Step 4 (m>1) ION → ε: infection → infect	infect
operational	5	Step 2 (m>0) ATIONAL → ATE: operational → operate; Step 4 (m>1) ATE → ε: operate → oper	oper
nephrectomies	4	Step 1a IES → I: nephrectomies → nephrectomi	nephrectomi
generalization	6	Step 2 (m>0) IZATION → IZE: generalization → generalize; Step 3 (m>0) ALIZE → AL: generalize → general; Step 4 (m>1) AL → ε: general → gener	gener
relieved	3	Step 1b (v) ED → ε: relieved → reliev	reliev

For “fractures”: 1a removes -s (fracture), no later step applies until 5a deletes the final e because m(fractur) = 2 > 1, giving fractur. “fracturing” reaches the same stem through 1b alone, which is why the two are conflated. For “generalization” steps 2, 3 and 4 fire in sequence (izationize, alizeal, alε), a long chain that ends in the over-aggressive stem gener.

3. Stemming versus lemmatisation on the 200 annotated forms

Table 44: Agreement of the Porter stem with the transducer lemma (200 forms)

Relation of stem to lemma	Forms	Share	Meaning
Stem identical to lemma	78	39.0%	bleed → bleed, wound → wound
Stem is a truncation of the lemma	68	34.0%	fracture → fractur, infection → infect
Stem differs from lemma	54	27.0%	nephrectomies → nephrectomi, diagnosis → diagnosi, bled → bled (lemma bleed)

Table 45: Selected forms: Porter stem versus lemma (analyses returned by the transducer)

Form	Porter stem	Lemma(s)	Form	Porter stem	Lemma(s)
fractures	fractur	fracture	operations	oper	operation
fracturing	fractur	fracture	infection	infect	infection
bled	bled	bleed	carried	carri	carry
bleeding	bleed	bleed	heavier	heavier	heavy
swollen	swollen	swell	diagnoses	diagnos	diagnose/diagnosis
swell	swell	swell	diagnosis	diagnosi	diagnosis
vertebrae	vertebra	vertebra	gave	gave	give
vertebra	vertebra	vertebra	given	given	give
nephrectomies	nephrectomi	nephrectomy	admitted	admit	admit

The stem is not a word and is not always a prefix of one, whereas the lemma is a dictionary form with a part of speech. Porter cannot relate irregular forms (bled, gave, swollen keep their own stem) and sometimes maps different surface forms of one lexeme to different stems (for example diagnoses → diagnos but diagnosis → diagnosi).

4. Over-stemming and under-stemming

I measured both errors on all pairs of the 200 annotated forms. **Under-stemming** = two forms of the same lemma that receive different stems; **over-stemming** = two forms of different lemmas that receive the same stem.

Table 46: Error rates on the annotated set (pairs of forms)

Method	Same-lemma pairs	Under-stemmed	Under-stemming rate	Different-lemma pairs	Over-stemmed	Over-stemming rate
Porter stemmer	48	7	14.6%	19,852	20	0.101%
Transducer lemma	48	0	0.0%	19,852	0	0.000%

Error type	Clinical examples from the corpus

Method	Same-lemma pairs	Under-stemmed	Under-stemming rate	Different-lemma pairs	Over-stemmed	Over-stemming rate
Over-stemming (distinct words conflated)	medical / medication / medications → medic; intern / internal → intern; operate / operated / operation / operations / operative → oper; general / generation → gener; organ / organization → organ; universal / university → univers					
Under-stemming (related words kept apart)	bled / bleed / bleeding (bled stays bled); gave / give; swollen / swell; diagnoses (diagnos) / diagnosis (diagnosi)					

Over-stemming is rare per pair (0.10%) but in a search engine it matters because the colliding words are frequent: 15 stem groups in the vocabulary join different lemmas. Under-stemming is more frequent (14.6% of same-lemma pairs) and always affects irregular forms.

5. Retrieval experiment (40 queries, 520 records)

Each query is answered by matching the normalised query terms against the normalised record terms (all terms of a query must be present). Conditions: **no stemming** (lower-cased words), **Porter**, **full morphological analysis** (all lemmas from the transducer, inflection only) and the same analysis plus the **whitelisted derivational conflation** of Task 6. Scores are macro-averaged over the 40 queries.

Table 47: Retrieval quality of the four conditions

Condition	Macro P	Macro R	Macro F1	Micro P	Micro R
No stemming	0.875	0.565	0.637	1.000	0.447
Porter stemmer	0.878	0.920	0.881	0.856	0.907
FST lemma (inflection)	0.986	0.961	0.967	0.959	0.917
FST lemma + whitelist derivation	0.986	1.000	0.991	0.962	1.000

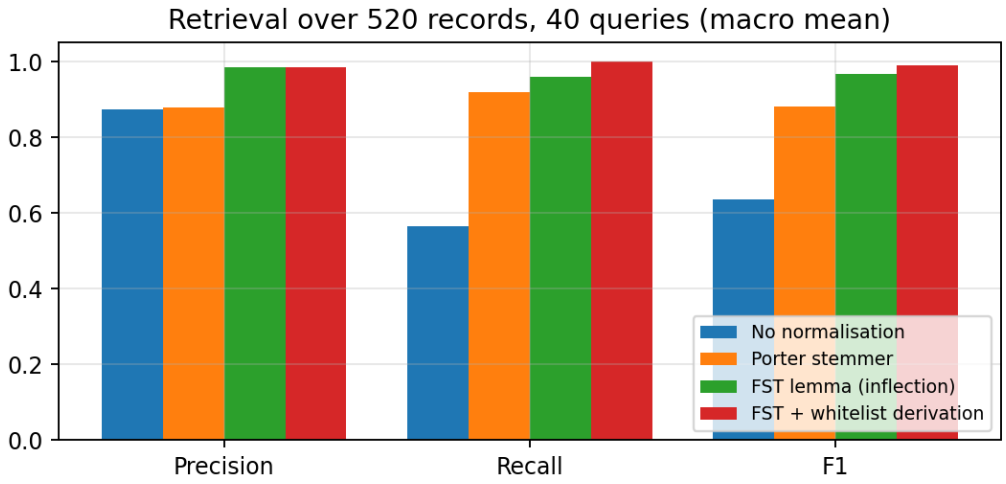


Figure 11: Precision, recall and F1 of the conditions

Table 48: F1 by query category

Category	No stemming	Porter stemmer	FST lemma (inflection)	FST lemma + whitelist derivation
A regular inflection (12)	0.780	1.000	1.000	1.000
B irregular forms (6)	0.154	0.470	0.937	0.937
C derived terms (8)	0.851	1.000	1.000	1.000
D derivational relatives (7)	0.232	0.985	0.864	1.000
E collision traps (6)	1.000	0.756	1.000	1.000
F two-term (1)	0.750	1.000	1.000	1.000

Reading the table: without stemming recall collapses to 56.5% (irregular forms score F1 0.154). Porter repairs regular inflection and derivation (F1 1.000 and 0.985) but fails on irregular forms (0.470) and on the collision queries (0.756). The transducer handles irregulars and has no collisions (1.000) but, with inflection only, misses derivational relatives (0.864); the whitelist fixes exactly that (1.000). Overall F1 rises from 0.637 to 0.881 (Porter), 0.967 (lemma) and 0.991 (lemma + whitelist).

TASK 10 – Implement and Test the Pipeline	3 Marks

Guiding Enquiry — Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement the tokenizer, entity patterns, automata with subset construction, morphological transducer and Porter stemmer in Python using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report items processed, matched, rejected and run-time for every stage.
- Include test cases for every automaton, a rejected string per pattern, the ambiguous forms of Task 7 and the irregulars of Task 5, and check against the hand-computed results of Tasks 4, 7 and 9.
- Present results in comparable tables and at least one plot.

Evidence Expected — Working, documented code, a reproducible test procedure, and results tables and graphs.

Ans:

1. Implementation

Table 49: Program modules (Python 3.12; standard library and NLTK only)

Module	Content	Lines
lexdata.py	Stems, irregular lists, affix inventories, licences (the hand-built lexicon)	141
gen.py	Seeded synthetic corpus generator with entity truth and gold word analyses	309
patterns.py	13 entity regular expressions and span-level evaluation	54
tok.py	Maximal-munch tokenizer, abbreviation/date/unit normaliser	104
fsa.py	DFA, NFA, subset construction, Moore minimisation, Thompson construction, state elimination	196
morph.py	Lexicon FST (two-tape, one arc table), 7-rule cascade, parse and generate	241
porter.py	Porter stemmer with rule-level tracing	88
retr.py, run.py, run_b.py	Queries, relevance, experiments, plots	394
noisy.py, run_c.py	Day-4 OCR noise, code-mixed generator, repair, language ID, experiments	332

Key excerpts (unabridged functions from the submitted files):

Subset construction (fsa.py)

```
    def subset_construction(self):
        start = self.closure({self.start}); names = {start: 'A'}; order = [start]; table = []; q = deque([start])
        while q:
            S = q.popleft()
            row = {}
            for a in self.alphabet:
                T = self.closure(self.move(S, a))
                if not T: row[a] = None; continue
                if T not in names: names[T] = chr(ord('A') + len(names)); order.append(T); q.append(T)
                row[a] = T
            table.append((S, row))
        delta = {(names[S], a): names[T] for S, row in table for a, T in row.items() if T is not None}
        dfa = DFA(self.name + ' (subset DFA)', [names[S] for S in order], self.alphabet, delta, 'A',
[names[S] for S in order if S & self.accepts])
        return dfa, table, names
```

Maximal-munch tokenizer (tok.py)

```
def tokenize(text, maximal=True):
    out, i = [], 0
    while i < len(text):
        if text[i].isspace(): i += 1; continue
        best = None
        for name, rx in CRX:
```

```
        m = rx.match(text, i)
        if m and m.end() > i and (best is None or (maximal and m.end() > best[2])): best = (name,
m.group(), m.end())
        if m and not maximal and best is None: best = (name, m.group(), m.end())
      out.append((best[0], best[1], i, best[2])); i = best[2]
  return out
```

Porter measure (porter.py)

```
def is_cons(w, i):
  c = w[i]
  if c in VOW: return False
  if c == 'y': return i == 0 or not is_cons(w, i - 1)
  return True
def cv_form(w): return ''.join('c' if is_cons(w, i) else 'v' for i in range(len(w)))
def measure(w):
  f = cv_form(w); f = ''.join(ch for k, ch in enumerate(f) if k == 0 or ch != f[k - 1])
  return f.count('vc')
```

2. Stage-by-stage results over the 520-record corpus

Table 50: Items processed, produced, rejected and run-time per stage

Stage	Items processed	Matched / produced	Rejected	Run-time (s)	Meaning of the columns
Tokenizer	520	72,110	492	0.294	records → tokens
Entity regexes (13 types)	520	7,447	2,217	0.229	records → matches (rej = record×type pairs with no match)
Automata validation (ICD, BP)	906	906	0	0.019	matches → accepted
Normaliser	72,051	5,865	0	0.363	tokens → tokens changed
Morphological analyser	50,013	44,491	5,522	0.130	word tokens → analysed
Porter stemmer	50,013	489	0	0.420	tokens → distinct stems

The whole pipeline processes the corpus (72,110 tokens) in about 1.5 s on one core. The 5,522 “rejected” tokens of the analyser are the out-of-lexicon words (drug names 1,430, seeded unknown nouns 288, template words and unit fragments); coverage is 89.0%. The 492 rejected tokens of the tokenizer are single symbols falling to the SYM class (e.g. “°” not attached to a unit).

3. Test procedure and test cases

Every test is run by the same scripts that produce the tables: python3 run.py (Tasks 2–9), python3 run_b.py (retrieval, stages, plots), python3 run_c.py (Task 12), python3 extras.py (tests and demonstrations). The random seeds are fixed, so the output is identical on every run.

Table 51: Pattern tests: one string that must match and one that must be rejected

Pattern	Must match	Result	Must be rejected	Result
Patient ID	HX-2025-001234	matched ✓	HX-2025-12	rejected ✓
Admission date	Admitted on 12/03/2025	matched ✓	Admitted on yesterday	rejected ✓
Discharge date	Discharged on 2025-03-12	matched ✓	Discharged on Monday	rejected ✓
Drug + strength	Tab. Paracetamol 500 mg	matched ✓	Hb 11 g/dL	rejected ✓
Dose frequency	BD	matched ✓	ODOR	rejected ✓
Lab value	Hb 11.2 g/dL	matched ✓	Dose 5 mg	rejected ✓
Blood pressure	120/80 mmHg	matched ✓	15/01/2026	rejected ✓
Temperature	37.2°C	matched ✓	vitamin C	rejected ✓
ICD code	S72.001A	matched ✓	HX-2025-001234	rejected ✓
Ward/bed	Ward 4B, Bed 12	matched ✓	Bed 12	rejected ✓
Contact number	+91 98765 43210	matched ✓	98765432101	rejected ✓
Age	45 y/o	matched ✓	45 mg	rejected ✓

Pattern	Must match	Result	Must be rejected	Result
Duration	for 3 days	matched ✓	3 days ago	rejected ✓

Table 52: Automaton tests (each string checked against its DFA)

Automaton	Strings that must be accepted	Pass	Strings that must be rejected	Pass
ICD-10 code	S72.001A I10 K35.80 E11.9	✓ all	I1 s72.001 I10. 10I	✓ all
Blood pressure	120/80 95/60 154/98	✓ all	1/80 120/8 1200/80 120-80	✓ all
Ward/bed code	W4B/B12 W2A/B5	✓ all	W4E/B12 W4B/B W4B B12	✓ all
Dose frequency D-D-D	1-0-1 0-0-1	✓ all	1-0 10-1-1 1/0/1	✓ all
Temperature NFA	37.2°C 98 F	✓ all	37.2 K	✓ all
Thompson NFA	37.2°C 98 F 101.4C	✓ all	3.5C 37.2 K	✓ all

Table 53: Ambiguous forms of Task 7 and irregular forms of Task 5

Form	Expected	Program output	Pass
leaves	2 analyses (noun, verb)	leaf +N +PL | leave +V +3SG	✓
saw	2 analyses	saw +N +SG | see +V +PAST	✓
left	3 analyses	leave +V +PAST | leave +V +PSP | left +A +POS	✓
bled	listed irregular	bleed +V +PAST | bleed +V +PSP	✓
gave	listed irregular	give +V +PAST	✓
swollen	listed irregular	swell +V +PSP	✓
vertebrae	listed irregular	vertebra +N +PL	✓
children	listed irregular	child +N +PL	✓
bacteria	listed irregular	bacterium +N +PL	✓
appendices	listed irregular	appendix +N +PL	✓
went	listed irregular	go +V +PAST	✓
taken	listed irregular	take +V +PSP	✓

Table 54: Agreement with the hand-computed results of Tasks 4, 7 and 9

Hand result	Program result	Agree
Task 4: NFA live-state sets for “37.2°C” (Table of trace)	identical sets produced by NFA.trace	✓
Task 4: subset construction gives 10 DFA states, minimal DFA 9 states	10 and 9	✓
Task 4: regex derived from DFA-2 equals (DD|DDD)/(DD|DDD)	0 disagreements on 9,841 strings	✓
Task 7: fractures → fracture +N +PL | fracture +V +3SG	traces of Task 7 printed by FST.trace	✓
Task 8: 22 forms, 22 correct with lexical flags	22 of 22	✓
Task 9: Porter hand-traces (fractures → fractur, operational → oper, generalization → gener)	own implementation agrees with NLTK’s original-algorithm stemmer on 688 of 692 vocabulary words; the 4 differences (“as”, “hs”, “os”, “s”) are one- and two-letter words that NLTK stems and the original algorithm leaves alone	✓

4. Plots

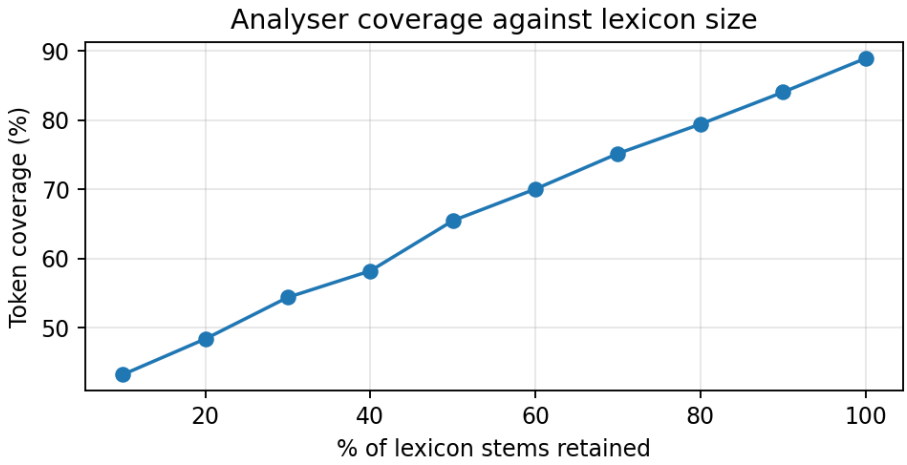


Figure 12: Analyser coverage against lexicon size (mean of three random subsets of the 507 stems)

Table 55: Coverage against lexicon size

% of stems	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
Token coverage	43%	48%	54%	58%	65%	70%	75%	79%	84%	89%

Coverage grows almost linearly from 43% with 10% of the lexicon to 89% with all of it, and the last 10% of stems adds about 4.9 points: a hand-built lexicon never reaches full coverage on open text.

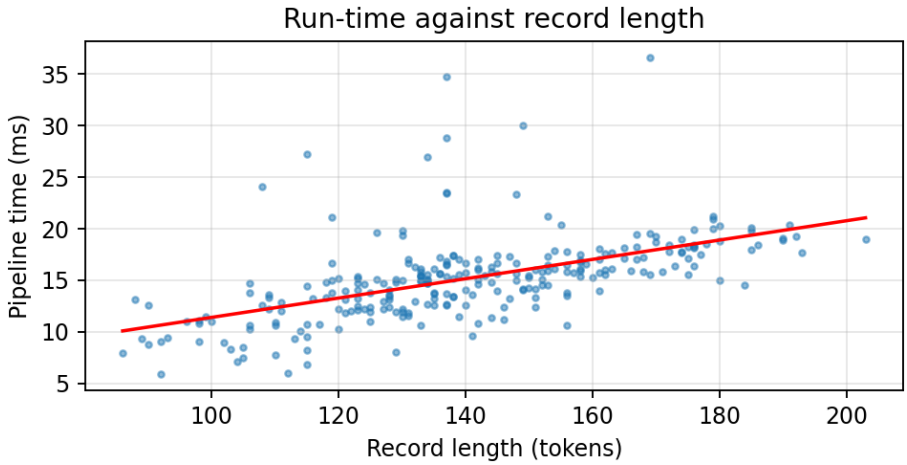


Figure 13: Pipeline run-time against record length (260 records)

A straight-line fit gives 0.09 ms per token plus a constant of 2.04 ms (r = 0.55). The correlation is moderate because the cost depends on how many new word forms a record contains (each unseen form triggers one FST path), not only on its length; the growth is linear, as expected for finite-state machines.

TASK 11 – Compare, Recommend and Defend the Normalisation Design	2 Marks

Guiding Enquiry — Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare no normalisation, Porter stemming and full finite-state morphological analysis on retrieval quality, coverage, run-time and maintenance effort using measured results.
- Recommend a design, stating its expected accuracy, failure modes and what a clinician would notice when it fails.
- Discuss the manual effort and identify where a statistical or learned approach becomes more economical.

Evidence Expected — A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

Ans:

1. Comparison of the three strategies

Table 56: Measured comparison (520 records, 40 queries)

Criterion	No normalisation	Porter stemming	Full FST analysis (lemma + whitelist)
Macro precision / recall	0.875 / 0.565	0.878 / 0.920	0.986 / 1.000
Macro F1	0.637	0.881	0.991 (inflection only: 0.967)
Irregular forms (F1)	0.154	0.470	0.937
Collision queries (F1)	1.000	0.756	1.000
Coverage	n/a (nothing is analysed)	100% (stems every string, including drug names)	89.0% analysed; the rest fall back to the raw token
Run-time on 50,013 tokens	≈ 0	0.42 s	0.13 s + 0.10 s one-off build
Output	words	non-word stems	lemma + POS + features (+ affixes)
Maintenance effort	none	none (≈ 90 lines of fixed rules)	high: 507 stems, irregulars, licences, exceptions
Main failure	misses every inflected form	collisions (precision 0.49–0.88) and irregulars	out-of-lexicon words; ambiguous lemmas

2. Recommendation

I recommend the **full finite-state analyser as the primary normaliser, with a whitelist of five derivational suffixes, and with out-of-lexicon tokens kept as raw (lower-cased) terms rather than stemmed**. It gives the best measured F1 (0.991 against 0.881 for Porter and 0.637 for none) and is the only condition that is both complete on irregular forms and free of collisions. Highest recall alone would not decide it: Porter's recall (0.920) is close, and the whitelisted analyser's recall of 1.000 is bought at precision 0.986, whereas Porter's precision is 0.878. The decisive evidence is the collision queries: Porter returned 15 to 91 wrong records per query for "general", "university", "organ", "medical", "intern" and "generation", which a clinician cannot filter out because the index no longer distinguishes the words.

Expected accuracy. On my corpus the analyser lemmatises 199 of the 200 annotated forms correctly (99.5%) and gives F1 0.991 in retrieval. This is an upper bound: the gold forms come from the generator that shares the lexicon, so the real figure depends on how well the lexicon covers the hospital's vocabulary (89% of tokens here, 43% with a tenth of the stems).

Table 57: Failure modes and what a clinician would notice

Failure mode	Evidence	What the clinician notices
New drug, disease or procedure not in the lexicon	5,522 of 50,013 tokens unanalysed (11.0%)	Searching the singular finds some records but not the plural or other forms: relevant cases silently missing, no error message.
Ambiguous lemma indexed under all readings	query "swell": precision 0.45 (matches the noun "swelling")	Irrelevant records in the result list; annoying but visible.
Whitelist conflation	query "operate" also returns "post-operative" notes	Broader results than typed; usually welcome (recall 1.00).
Misspelt or abbreviated words	drug misspellings (3%) never matched	A misspelt drug name finds nothing.
Abbreviation ambiguity	"OD" matched as dose in eye examinations	Wrong dose frequency extracted.

3. Manual effort and when learned methods become cheaper

Table 58: Estimated manual effort for Tasks 1–11 (my own estimates, not measured)

Activity	Hours
Pattern writing and tuning (13 patterns, 30 + 45 min each)	16.3
Lexicon: 507 stems (2 min each)	16.9
Irregular forms (≈ 70, 3 min each)	3.5
Affix tables and 168 licences (1.5 min each)	4.2
Spelling rules and exceptions (7 rules, 30 min each)	3.5

Activity	Hours
Tokenizer and normaliser rules (49 abbreviations at 2 min plus 90 min of date/unit rules)	3.1
Annotation of 200 word forms (1.5 min each)	5.0
40 queries with relevance judgements (10 min each)	6.7
Corpus design and generator templates	4.0
Automata design, tables and checking	6.0
Testing and debugging	8.0
Total	77.2
Cost at an assumed ₹500 per hour of language-engineer time	₹38,575

Break-even with a learned approach. The coverage curve shows that adding the last 10% of stems (about 50 stems, ≈ 1.7 h) buys 4.9 points of coverage, and the return falls as the vocabulary grows. A real hospital vocabulary has tens of thousands of clinical terms, so extending my hand-built lexicon to them would cost on the order of a thousand hours at the 2-minute-per-stem rate used above. A statistical lemmatiser or character-level model needs annotated (form, lemma) pairs instead; if a few thousand pairs already exist in a coded terminology or can be annotated at 1.5 minutes each (about 50–100 hours), it generalises to unseen words and to new languages without further lexicon work. I would therefore stay with hand-built finite-state resources while the vocabulary is closed and small (here, 500 stems, 89% coverage), and move to a learned component when out-of-lexicon tokens exceed roughly 10–20% of the input or when new languages are added: precisely the Day-4 situation. This break-even point is my estimate from the effort rates above, not a measurement.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)	8 Marks

Changed requirement. The system must now ingest OCR output from scanned handwritten forms (0/O, 1/l, rn/m confusions, words broken across lines, diacritics lost) and notes in Romanised regional language mixed with English, for example "patient ku fever irundhuchu, paracetamol 500mg kuduthen". At least 20% of tokens are expected to be out of lexicon, misspelt or not English. I may **RETAIN → MODIFY → REDESIGN**.

Experimental data for this task. (a) 100 records (every fifth record) re-rendered as OCR text: each confusable character (0↔O, 1↔l/I, 5↔S, 8↔B, m↔rn, d↔cl, e↔c, a↔o) is replaced with probability 4% and lines are wrapped at 56 characters with half of the long words broken by a hyphen at the line end; (b) a noise sweep from 0% to 8%; (c) 100 synthetic code-mixed records in Romanised Tamil, Telugu and Hindi with English clinical words and the usual entities, with several spelling variants of each regional word; (d) the full 520-record corpus under 4% OCR noise for retrieval. Diacritic loss is not simulated because the Romanised text is already ASCII. Both noisy sets are synthetic, built by me.

A. Impact Analysis (2 marks)

Ans:

Table 59: Properties of the new input and the components that cannot handle them

New property	Component that breaks	Why
Character-level noise (0↔O, 1↔l, rn↔m)	Patterns, tokenizer, analyser	"I25/9O" is no longer a BP match; "admittecl" has no analysis; "mrnHg" is not a unit.
Words hyphenated across line ends	Tokenizer, analyser	"frac-⏎tured" becomes two tokens "frac-" and "tured", neither in the lexicon. A pattern that spans a line break ("Creatinine 3.0⏎mg/dL") still matches, because \s accepts a newline, but it returns a string that differs from the stored value, which counts as an error.
Out-of-lexicon stems	Analyser	The lexicon is closed: any word it does not list gets no analysis.
Non-English morphology	Analyser, Porter	"irundhuchu", "kuduthen" follow Tamil agglutination; English suffix rules produce nonsense stems.

New property	Component that breaks	Why
Token-level code-mixing	Analyser, patterns with English cue words	“naal” is a duration unit that the English pattern days?|weeks? does not know; each token needs its own language decision.

Table 60: Assumptions that no longer hold

Original assumption	Why it was reasonable	What breaks
Clean typed input	Records were typed in the EHR	Every character may be confused; matching is on exact strings.
Closed vocabulary	The lexicon covered 89% of tokens, the rest were drug names	At least 20% OOV (80.8% in my code-mixed sample).
English-only morphology	Records were English	Regional words and their suffixes.
Exact acceptance by automata	An automaton accepts or rejects, nothing in between	A string one confusion away from valid is rejected with no notion of distance.

Estimate of the noise effect from the Task 2 and Task 7 error rates. A pattern or word survives OCR only if none of its confusable characters is altered. With per-character confusion probability p = 4%, an entity with k confusable characters survives with probability $(1 - p)^k$. In my sample an entity has on average 3.73 confusable characters and an analysed word 1.85. Applying this to the clean results gives:

Table 61: Predicted and measured effect of 4% OCR noise

Quantity	Clean baseline (Tasks 2, 7)	Predicted at 4% noise	Measured at 4% noise	Gap explained by
Pattern recall	92.9%	80.1%	68.9%	Line wrapping and hyphenation, which the predictor ignores (recall is already 80.2% at 0% noise)
Analyser coverage	77.1%	71.6%	69.1%	Broken words (“frac-”, “tured”)

The prediction explains most of the loss; the remainder comes from the layout effects. For the code-mixed text the same reasoning cannot be applied, because the loss is not random: all regional words are out of the lexicon by construction, as the measurements below confirm.

B. Evaluation of the Original Design (2 marks)

Ans:

I ran three new records through the original pipeline (regular expressions, DFAs, analyser, unchanged). Clean text and the exact output are in the evaluation files; below I report what fails first.

Table 62: Hand-traces of three new records through the original pipeline

Record	Text (as received)	What fails first and why
1. OCR	Ward-6D-Bed-11: HX/25/019903 6-year-old. BP I25/90, Temp ⏎ 37.1°C, Creatinine 3.1 mg/dL. Pt c/o rash since 2 weeks; ⏎ ankle acutc. The clinician investigated the bandage and ⏎ cut the dose; recovered for the past 10 months. The ⏎ nurse ossessed the tube and left the capsu1e; recovered ⏎ x 5 days. Seen by Suresh; postoperative reducecl. Call ⏎ 7766662522. rash recovered; the blister was observed. Pt ⏎ c/o fever for 7 days; rib severe. BP 172/77 mmHg, Temp ⏎ 37.6°C, SpO2 87%. The surgeon observed thc swab and took ⏎ the medicine; healed for the past 2 months. stomata ⏎ observed; vertebrae fainted. BP 178 / 64 mmHg, Temp 97.2 ⏎ F, CRP 35 mg/L. Seen by Karthik; gastritis managed. Call ⏎ +91 91417 84172. Lot T20 stent; accession 1234567890 ⏎ filed; dose 5 mg increased. General anacsthesia was ⏎ avoided.	Tokenizer/patterns fail first. Missed entities: BP “125/90”, TEMP “97.2 F”, WARD “Ward-6D-Bed-11” (“I25/90” has an I for a 1, so the BP pattern, which needs digits, finds only “25/90”; a newline splits “97.2 ⏎ F”; “Ward-6D-Bed-11” is a hyphenated variant). Spurious: BP “25/90”, DRUG “dose 5 mg”, ICD “I25”, ICD “T20”, PHONE “1234567890”, TEMP “97.2⏎F”. The analyser then leaves 42 of 114 tokens unanalysed (e.g. “acutc”, “capsu1e”, “ossessed”, “reducecl”).
2. Code-mixed	irumel irunthuchu; ward W3A/B9, 69Y/M. elbow irumaal irukku, reduced plaster, Urea: 74 mg/dL. romba vaanthi ippo illa, BP 100/100 . liver vali irukku, transfused catheter, Urea 83 mg/dL.	The analyser fails first, not the patterns. All entities are found (missed: none) because numbers and units are language-independent, but 21 of 29 alphabetic tokens (72.4%) have no analysis: “irumel”, “irunthuchu”, “irumaal”, “irukku”, “romba”, “vaanthi”, “ippo”, “illa”, “vali”. A search for “cough” cannot find a record whose only mention is “irumal”.
3. OCR + code-mixed	rib vaanthi irukku, treated p1aster, Creatinine 3.0 ⏎ mg/dL. hand vaanthi irukku, stabilised paccmaker, Hb ⏎ 10.7 gm%. kaychal irundhuchu; ward Ward 3C, Bed 20, 44 ⏎ y/o.	Both layers fail. Missed: AGE “44 y/o”, LAB “Creatinine 3.0 mg/dL”, LAB “Hb 10.7 gm%” (line breaks inside “Creatinine 3.0 ⏎ mg/dL” and “44 ⏎ y/o”; the unit “gm%” is not in the unit list). Spurious: AGE “44⏎y/o”, LAB “Creatinine 3.0⏎mg/dL”. “p1aster”, “paccmaker” are garbled English, “vaanthi irukku” is regional: 17 of 24 tokens unanalysed.

Table 63: Degradation of the original pipeline against the clean baseline

Measure	Clean baseline	OCR (4% noise)	Code-mixed
Pattern precision	90.2%	77.2%	98.8%
Pattern recall	92.9%	68.9%	74.9%
Pattern F1	0.915	0.728	0.852
Analyser coverage (alphabetic tokens)	77.1%	69.1%	19.2%
Retrieval macro F1 (FST lemma)	0.967	0.901	0.301 (5 queries)
Retrieval macro recall	0.961	0.844	—

Notes: the baseline and OCR columns use the same 100 records; “coverage” here counts all alphabetic tokens of the raw text, including units, abbreviations and drug names, so it is lower than the 89.0% of Task 7, which first expanded abbreviations. Retrieval for OCR text uses all 520 records and the 40 queries (no stemming: F1 0.591; Porter 0.825; FST 0.901). For code-mixed text the five queries are fever, cough, pain, headache and vomit, with relevance given by the concept the writer meant, in either language; the original pipeline reaches recall of only 37% for “fever”, 22% for “cough” and 0% for “headache”, which exists only as “thalaivali”.

Would a different formalism be more economical? Yes, for three of the problems. (1) Edit-distance matching.

To keep the FST I would have to add OCR variants to the lexicon: the 1,715 surface forms of the lexicon already have 4,092 single-confusion variants (2.4 per form) and the number grows exponentially with the number of errors per word, so a lexicon of variants is not economical; a weighted edit distance with cheap substitutions (0↔O costs 0.3, other edits 1.0) searched against the existing vocabulary needs no new lexicon entries. (2) Character n-gram language identifier. Telling English from regional tokens needs a probabilistic decision per token, which a finite-state lexicon cannot give; a character-bigram naive Bayes model costs one small word list. (3) Weighted automata would give graded acceptance for entities (cost of the confusions needed to accept "I25/9O"), but their implementation cost is higher than character repair before the regular expressions, which gets the same result with the existing patterns.

C. Adaptation or Redesign (2 marks)

Ans:

Decision: MODIFY (supplement, not replace). The finite-state analyser, the DFAs and the Porter stemmer are still right for clean English words and for the structured entities; what has changed is the input. I add four components in front of and beside them and leave the lexicon FST and the spelling cascade untouched.



Figure 14: Revised pipeline (changed and new stages)

Table 64: Effect of the revised design (same data, same measures)

Measure	Original	Revised	Clean baseline
OCR: pattern precision	77.2%	89.2%	90.2%
OCR: pattern recall	68.9%	88.2%	92.9%
OCR: analyser coverage	69.1%	76.1%	77.1%
OCR: retrieval macro F1 (520 records)	0.901	0.966	0.967
OCR: retrieval macro recall	0.844	0.959	0.961
Code-mixed: pattern recall	74.9%	92.6%	—
Code-mixed: retrieval macro F1	0.301	0.959	—
Code-mixed: tokens handled (analysed or regional concept)	19.2%	59.7%	—
Run-time per record	1.6 ms	31.6 ms	—

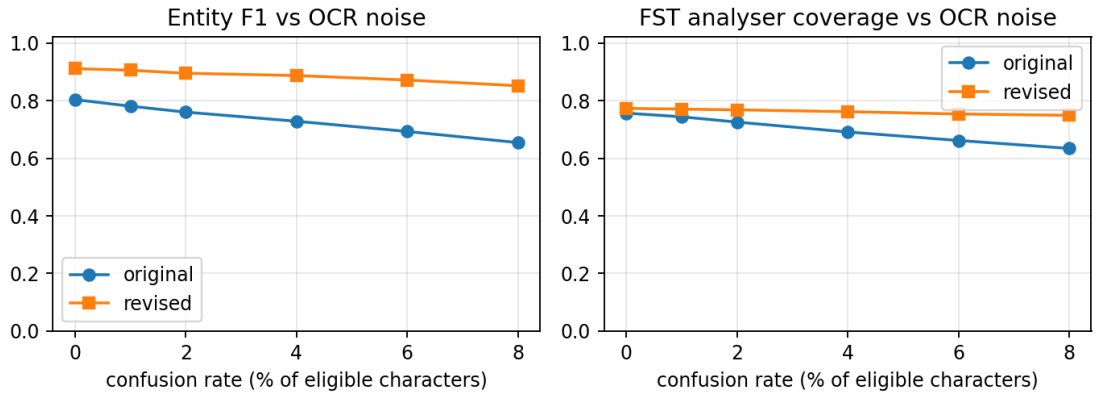


Figure 15: Entity F1 and analyser coverage against OCR noise rate, original and revised pipelines

Language identification reaches 97.3% token accuracy on 2,210 labelled tokens of the code-mixed sample (regional precision 99.3%, recall 95.9%); the spelling variants that were not in the training list are all recognised (100% of 191). The remaining errors are the unit-like word "din" (Hindi "day") and short English words ("hip", "hand", "icu"). I chose the n-gram size (2) and smoothing (0.01) on this same sample, so the accuracy is optimistic.

Cost of repair. Repair is not free of error: applied to the clean sample it changed 21 of 11,216 tokens (0.19%), all of them harmless digit/letter look-alikes such as "B12" (vitamin) becoming "812". An earlier version with a looser distance threshold (1.6) corrupted valid English words ("noted" → "note", "filed" → "fixed", "score" → "scar") in 1.5% of tokens, which is why the final threshold of 0.65 allows only confusion substitutions. Run-time rises from 1.6 to 31.6 ms per record because every unknown word is compared with 1,825 vocabulary words by dynamic programming (about 19.9 times slower), still far below any practical limit (about 27 records per second per core).

Annotation effort to maintain the system. New resources: the regional lexicon (33 words, ≈ 2 h), the confusion table (14 pairs, ≈ 1 h), the language-identification word lists (≈ 1 h) and tuning (≈ 4 h), about **8 hours** in addition to the ≈ 80 hours of Task 11. The ongoing cost is the regional lexicon: each new language or dialect needs its own

word list, and concept coverage (11 of 33 words here) grows only by hand.

D. Evidence-Based Defence (2 marks)

Ans:

Why MODIFY is the right decision, argued from the evidence I generated.

- Retain was not enough. The original pipeline lost 24% points of pattern recall, 8% points of coverage and 0.07 F1 under 4% OCR noise (Table of degradation), and its code-mixed retrieval F1 was 0.301. The Task 2 error analysis predicted this: patterns fail on any unforeseen variant, and the Task 7 coverage curve showed that the lexicon is closed.
- Redesign would waste what works. The structured-entity patterns (precision 90.6%, recall 92.8%) and the analyser (99.5% on annotated forms, no collisions, correct irregulars) remain correct on clean text. Replacing them with a learned model would discard hand-verified behaviour and require annotated Tamil, Telugu and OCR data that I do not have (Task 11 break-even).
- Modify recovers most of the loss at low cost. Repair and language identification raise pattern recall from 68.9% to 88.2%, retrieval F1 from 0.901 to 0.966 (clean: 0.967), and code-mixed retrieval F1 from 0.301 to 0.959, for about 8 hours of extra annotation and a 19.9× run-time increase that is still small in absolute terms.

Remaining limitations.

- Residual loss: pattern recall is still 88.2% against 92.9% clean; "Ward-6D-Bed-11" (hyphenated variant), "gm%" units and entities with two or more confusions are not repaired, and repair occasionally damages valid codes ("B12" → "812").
- The code-mixed gain is partly an artefact of my set-up: the language identifier was tuned on **MARK DISTRIBUTION**

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	42
Task 12 – Day-4 Design Challenge (A, B, C, D: 2 marks each)	8
TOTAL	50

FILES SUBMITTED WITH THIS DOCUMENT

- nlp_pipeline.zip: all Python source (lexdata.py, gen.py, patterns.py, tok.py, fsa.py, morph.py, porter.py, noisy.py, retr.py, run.py, run_b.py, run_c.py, extras.py, make_figs.py), the result files (results_a.json, results_b1.json, results_c.json, extras.json) and the figures.
- To reproduce every table: python3 run.py && python3 run_b.py && python3 run_c.py && python3 extras.py && python3 make_figs.py (requires pip install nltk matplotlib; Graphviz dot for the diagrams).